Full Stack:-

1. Combination of Frontend and Backend
2. Combination of multiple pages is called a website

3. **Mern:-** Combination of Frontend and Backend with the frameworks and libarires

1. **M->**MongoDb
2. **E->** Express :- it is a frame work for node js
3. **R->**React:- it is a js libarary
4. **N->**NodeJs

Technologies in the frontend:- Html (hyper text markup language), css(cascading style sheet), bootstrap(framework), js (javascript) -> ts (typescript)framework,react(libarary)

Technologies in the backend:- nodejs -> expressjs as a framework, mongodb(backend)

5. Collection of modules is known as modules
6. Framework:-getting from another modules or another projects and using them in our current project

Html:- is a skeleton of the webpage, latest version is html5,it is a structure/layout

CSS:-it is a style

JS:-Logic

Html5:-

Structure:-

1. **<!DOCTYPEhtml>:-** We are saying that our website is in the version5 and it is a document type
2. **<html>:-**It is a root tag, it is the parent of head & body tag
3. **<head>:-**We will keep this tag at the top of the html, we have title inside this
4. **<body>:-**body at the below of the header tag, inside we can write headings,paragraphs,imgs,videos,iframes….etc everything we will write here

Basic Html Page Example:-

```
<!DOCTYPEhtml>   //tells browser you are using 5 version
<html>
    <head>
        <title>My First Page</title>
    </head>
    <body>
        <p>Hello World</p>
    </body>
</html>
```

1. Extension is .html
2. Type (!) symbol and press enter we get the default html code it is possible only in the vs code
3. In the default code en means English language
4. Device width means it should fit to the device width
5. To edit html we can use inspect, view elements
6. Html is not case sensitive but writing in small letters is better for good programmers ex:- <html>= <HTML>

Inspect:-

1. Go to the chrome browser-> right click->inspect and hover on your browser you can observe the code will be get changed, it is possible to change the code
2. For example if you change the placeholder in the code in the position of google search icon you changed that placeholder to Search what you want you will get that change the google search icon it is the temporary thing
3. After closing the inspect and clicking on the refresh button it will be changed to normal
4. We can easily fix the bugs by using this inspect

Comment:-

1. Hide the data from the website
2. <!--don't want to render this on the website -->

Html Tag:-

1. A container for some content or other html tags
2. Ex:- This is a Paragraph here inside the tag it is a content and whole this is combined to call as element

Html Attributes:-

1. Attributes are used to add more information to the tag
2. Ex:- <html lang=”en”>

Heading tags:-

1. Contains 6 tags from h1 to h6 ,h1 is the most important tag and h6 is the least important tag

Live Server:-

Open vsc -> go to the extensions symbol it is 3rd symbol from down an type live server you will get like as below and click on install in the first one

After Completion of installation click on the GoLive below you have in the last in red color

After completion we will get the port no there ,in the below image I have the port no like 5500

2. You can check the output in the chrome browser with the port no like as the below image

3. In the paragraph tag we can type lorem inside the paragraph to get the default paragraph content

For autosave in vsc:-

4. Click on the file and below you can see the autosave we don't need to save everytime and we don't need to refresh in the output screen everytime by using this

Anchor tag:-

5. Simply write a and enter you will get the default code
6. Used to add link to your page
7. YouTube for opening youtube we can keep any link inside href
8. To open the youtube in the new tab use target => target="_main"/ target="_blank"

Img Tag:-

- This is also a attribute used to keep images

We have 2 types

- **Keeping direct image link :-**for this go to the browser and select any img -> right click-> click on the copy img link and paste in the src of img tag while coping we will get link in multiple lines it is not good for programmers so use 2nd type

- **Give the path from your folder:-**for this go to the browser and select any img -> right click-> click on the save image as it will redirect to our desktop then save where you want
-

Basic Html Tags:-

<bold>:-by default it has some font weight <bold>Attuluri</bold>

Prudula
<u>Sri</u>
<big>This is Big Tag</big>

:- To break the line/to enter into new line

<small>This is Small Tag</small>

_{Subscript:-H2O}

^{Superscript:-H2o}
<pre>:- Used to display the text as it is without neglecting even single space and next line

Ex:- <pre>This

```
        is
        a
        sample
        pre tag text
    </pre>
```

Semantic tags:- The name itself contaning a meaning

Ex:- <header>,<main>,<footer>

<section>:-For a section on your pg

<article>:- For an article on your pg

<aside> for content aside main content(ads)

Div Container:-

- Container means takes some width and height
- Div is a container used for other html elements

- This is block element takes full width
- We can write multiple div's inside one div

Span:-

It is an inline element

Diff b/w div and span ➔ div is an block element, span is an inline element

Lists:-

We have three types of lists

- Ordered list
- Unordered list
- Definition list

Ordered list:-

Syn:-

```
<ol>
    <li></li>
    <li></li>
</ol>
```

Unordered list:-

Syn: -<ul>

```
    <li></li>
    <li></li>
</ul>
```

Tables:-

Used to represent real life table data

<tr>:-used to display rows

<td>:- used to display data

<th>:-used to give column name

Ex:-<table>

```
    <tr>
        <th>Name</th>
        <th>RollNo</th>
    </tr>
    <tr>
        <td>Prudula</td>
        <td>21</td>
    </tr>
</table>
```

<thead> To wrap table head

<tbody>To wrap table body

Iframe:-

Website inside a website

Syntax:-<iframe src="https://www.youtube.com/watch?v=usymjvyy_mM" frameborder="0">Link</iframe>

Frame border is nothing but adding border

Type frame and click on enter to get default code

To get the output as our expectation

We want to add embed in the link

For this we want to change https://www.youtube.com/watch?v=usymjvyy_mM to `https://www.youtube.com/embed/usymjvyy_mM`

Day:-2 of Mern Full Stack

Forms in Html:-

9. Forms are used to collect the data from the user
10. Developers are also the users, because they are also using their website
11. Ex:- sign up/login/help requests/feedback/login/registeration/contact me

Syn:-

```
<form>

    Form content

</form>
```

Action in Form:-

12. Action attribute is used to perform an action after clicking on the submit in the form

13. Form Elements: Input

14. **Syn:-**<input type="text" placeholder="Enter Name"/>
15. What ever we are writing that is known as text like alphanumerics and special characters, when we give text it will gives the input bar to enter the text,inside the input we will have placeholder
16. We have types like mail,number,date…etc

Label:-

17. Writing name and value are not compulsory
18. In radio we can able to select only one option

Radio:-

```
<label for="id1">
    <input type="radio" value="classx" name="class" id="id1">
</label>

<label for="id2">
    <input type="radio" value="classx" name="class" id="id2">
</label>
```

19. Ex:-Gender, Branch

CheckBox:-

```
<label for="id1">
    <input type="checkbox" value="classx" name="class" id="id1">
</label>

<label for="id2">
    <input type="checkbox" value="classx" name="class" id="id2">
</label>
```

20. We can able to select multilple options ex:- hoobies,skills ,languages

TextArea:-

We can able to write 1000 of lines ,the data will be in rows and columns

```
<textarea rows=5 cols=10></textarea>
```

Select:-

It is like dropdown

```
<select name="city" id="city">
    <option value="Delhi"> Delhi</option>
    <option value="Mumbaii"> Mumbaii </option>
    <option value="Banglore"> Banglore </option>
</select>
```

Video Tag:-

Syn:-<video src="myVid.mp4">My Video</video>

Attributes:-

21. **Controls:-**we will get playbutton and stop button and pause button,we can control where to start and where to move
22. **Height**:-give height of the video you want to get on your website
23. **Width** :-give width of the video you want to get on your website
24. **Loop: -**plays infinite of times when we open a website
25. **Autoplay:-** that video will stop after completion of the video

26. **muted**

Website for better videos:-pixaboy we have the high quality websites

Max height and width of the video is 1920x1080, make sure that is less than 10mb or 20mb

Ex:- `<video src="./videos/video.mp4" height="300px" width="300px" controls autoplay muted/>`

Icons:-

27. For icons use font awesome website there go last and click on the get started there you can see lot of icons and styles
28. Before using the icons you want to import them
29. For importing we want to copy link form the cdns(font awesome libarary) website we will select first link there because it contains all the styles at last we have all.min.css
30. We will paste that link inside the head tag
31. For that type link and click enter you will get default code in that inside the href you can paste this link
32. After pasting link directly go to the font awesome page click on icons and search for what icons you want
33. If you want to redirect to the particular page when you click on that you need to keep that icon code inside the anchor tag

CSS(CASCADING STYLE SHEETS)

- Css describes how html elements are to be displayed on screen,paper, or in other media
- .css is the extension of the css

Types of Css:-

34. **Inline:-**directly include in the tag
    Ex:- <h1 style=”color:blue;text-align=”center”></h1>

35. **Internal :-**we will keep inside the head tag
    Ex:-<head>
    <style>
    Any styles
    </style>
    </head>
36. **External :-** we need to create a new page and we will keep link to the html page
    Ex:-<link rel=”stylesheet” href=”filename.css”>

Selectors:-

Syn:-Selector {property :value;}

Ex:- h1{color:blue;}

- **Universal selector (*):-**

  Syn:-*tagname{

  Style Here proprties

  }

- **Element Selector:-**
  Syn:- tagname{
  Style Here proprties
  }

- **Class selector (.):-**Class attributes specifies one or more classnames for an element
  Syn: - .tagname{

  Style Here proprties

  }

- **Id Selector (#):-**To maintain uniqueness we will use this

  Syn:- #tagname{

  Style Here proprties
  }

- **Combinator Selector:-Combination of two or more selectors,seprate by (,)**

Important CSS Properties:-

- **Font-style**
- **Font-Weight:-**increase the boldness of the weight
- **Text-decoration- 3types underline,overline,linethrough**
- **Border**:-how much piexels we want,dashed/solid/dotted,color

Ex:- border:solid;
border:2px; (or) border:solid 2px blue; (shorthand)
border:red

- **Border-width:-** how much width you want to specify for the border,it is nothing about the border thickness
- **Border-color**
- **Margin:-** we have margin-left, margin-right,margin-top,margin-bottom same for padding also we have, outer side of the border is the margin
    - Space between one content to another content
    - If we give margin top it will moves to the bottom
- **Padding:** space between the content and the border,border will be same only the content will move to left/right/top/bottom
    - Shorthand for margin/ padding:- margin/padding:top right bottom left
- **Color:-**color for the text we don't have any font or text color
- **Background-color**
- **Cursor:-**we will get hand symbol,when we hover on that,we have values like move, crosshair, pointer
- **Font-size:-**values like 10px,20px..etc
- **Font-family**
- **Height:-** values like 100px,40%..etc ,100% means it takes the whole percentage of the image
- **Width**
- **Background-image**:- url('link address');
- **Background size**:- mostly we use cover, to cover the image for whole width and height
- **Border-radius:-**to maintain curves for the border
- **Font-style:-**normal,italic,oblique

Note:-Box Model is important topic for the interviews

<u>Psuedo elements:-</u>

Syn:-Tagelement::hover{
Style
}

We have values like as below
::first-letter
::First-line
::Before

::After
::selection

CSS Float:-

The float property is used for positioning and formatting content

e.g:- let an img float left to the text in a container

- **Left:-**the element floats to the left of its container
- **Right:-** the element floats to the right of its container
- **None:-**

To display the paragraph to the particular right/left side we use this

DAY:-3 Of MERN Full Stack

CSS Positioning:-

The position property specifies the type of positioning method used for an element

There are 5 diff position values

37. **Static:-**default value
38. **Relative:-** of we want to keep anything in the middle we want to write top, left, bottom, right 0 value it will come to the center
39. **Fixed:-** If we want to keep in one fixed position we will use this, if we move the content this is also moves
40. **Absolute:-** To keep something on the top
41. **Sticky:-**it will stick to the top if we give top, if we move down it will doesn't change it will stick to the top

Z-Index:-

42. The Z-Index property specifies the stack order of an element
43. Z-Index only works on positioned elements (position: absolute, relative, fixed, static, sticky)
44. We mostly use this to get the content2 on another content1 like paragraph on the image , for this want to give less value for paragraph and more value for the image

OverFlow:-

Visible:-Default, The overflow is not clipped, the content renders outside the element's box

Hidden:-the overflow is clipped, and the rest of the content is invisible

Scroll:-the overflow is clipped, and a scrollbar is added to see the rest of the content

Auto:-Similar to scroll, but it adds scrollbars only when necessary

Css Display:-

None:- It will not display on the website

Inline:- It will block the content

Block:- The content will goes outside of the content

Inline-block:- it blocks the content in that line only like only inside of the card

Css Grid:-

45. It offers a grid-based layout system, with rows and columns, making it easier to design web pages without having to use floats and positioning

Syn:- display: grid; or display: inline-grid

46. The spaces b/w each column/row are called gaps
47. You can adjust the gap size by using one of the following properties:
48. **Column-gap:-**if you want gap only between the columns you can use , we want to specify the value in terms of pixeles
49. **Row-gap:-** if you want gap only between the rows you can use , we want to specify the value in terms of pixeles
50. **Gap**

Breakpoints:-

How your website will behave across device or viewport sizes

51. **Extra Small Devices :-** (Xs) in bootstrap <576px (device size)
52. **Small Devices:-** (s) in bootstrap >=576px (device size)
53. **Medium:-** (m) in bootstrap >=768px (device size)
54. **Large :-(**l) in bootstrap >=992px (device size)
55. **Extra Large:-** (xl) in bootstrap >=1200px (device size)

ShortCuts:-

56. .classname and click on enter to get the div default tag
57. For creating multiple divs we have a short cut **div*(n)** ➔ n means no of divs you want
58. For commenting we have shortcut ctrl + forward slash

Psudo-class:- stores in var only

- We will set css theme by using this

```
:root{
      --background-color:#8bc0f8;
      --text-color:white;
      --box-shadow:0 2px 4px rgba(0,0,0,0.1);
      --border-radius:10px;
      --spacing-unit:8px;
```

```
}
```

- While using pseudo classes and setting theme we will keep – at the starting of the line
- For using the above pseudo classes we want use them like as below

```
body{
    font-family: 'Courier New', Courier, monospace;
    line-height: 2;
    background-color: var(--background-color);
    color: var(--text-color);
}
```

Media Queries:-

EX1:-

```
@media (minwidth:600px){
                //this will be applicable for only extra small and small
}
```

EX:-2

```
@media (minwidth:800px){
                //this will be applicable for only extra small and small and medium
}
```

EX:-3

```
@media (minwidth:1000px){
                //this will be applicable for only extra small and small and for large
}
```

@media for importing the things of responsiveness

Structure:-

```
@media(min-width:value in px){
                .classname{
                        prop:val
                 }
}
```

For Example:-

```
body{
                background-color:black;
}
```

```
@media(min-width:600px){
                body{
                        background-color:white;
                 }
}
```

Animations:-

Hover:-

Pulse

Floating

Tossing

Fade

Grow:-Zoom in , Zoom out

Circle

Curls

Bubble

Light

Entrances:-

Slide down

Slide up

Slide left

Expand

Fade-in

Fade-out

Entrance

Day-4 of Mern Full Stack

Copy link form bootstrap5 :- for this open bootstrap5 in chrome or Microsoft edge after opening scoll down upto the starter template copy the code and paste in your folder

BootStrap:-

We have mainly two types of layout methods:

59. Flexbox (Stable)
60. CSS Grid (Unstable)

Flexbox:-

Properties:-

61. We will keep all in the classes in html file
    1. **Flexbox container:-** d-flex
    2. **Direction:-**specifies to row or column , flex-row or flex-column
    3. **Justify content :-** it will moves to the center or left or right in the same row, justify-content-start, justify-content-center, justify-content-start, justify-content-end
    4. **Align items:-**text-center, text-left
    5. **Wrap**
    6. **Wrap content**

Bootstrap Grid System:-

62. It uses a series of containers, rows, and columns to layout and align content.
63. Its built with flexbox and is fully responsive
64. It mainly works with
65. Container
66. Row
67. column
68. Here every grid contains 12rows and 12 columns, even we take a small part it also contains 12 rows and 12 columns
69. First we want to wrap everything in the container and then in the row and then column
70. **col-12** it is applicable for all the devices
71. **col-sm-12** only for small devices

72. **col-md-12** for medium devices
73. **col-lg-12** for large devices
74. **col-xs-12** for extra large devices
75. **col-xl-12** for extra extra large devices

Margin Prefixes in bootstrap:-

76. **margin:**m-
77. **margin-right:** mr-
78. **margin-left:** ml-
79. **margin-top:** mt-
80. **margin-bottom:** mb-

Margin size & values:-

81. Size range:0-5
82. 0 :- 0
83. 1 :- 0.25* spacer
84. 2:- 0.5 * spacer
85. 3 :- 1* spacer
86. 4 :- 1.5* spacer
87. 5 :- 3* spacer

Here spacer:-16px

Auto:-

88. It checks the respective parent element and specify the margin
89. We have like as below
90. m-auto
91. ml-auto
92. mr-auto
93. mt-auto
94. mb-auto

Padding Prefixes in bootstrap:-

95. **padding:**m-
96. **padding -right:** mr-
97. **padding -left:** ml-
98. **padding -top:** mt-
99. **padding -bottom:** mb-

Padding size & values:-

100. Size range:0-5
101. 0 :- 0
102. 1 :- 0.25* spacer
103. 2:- 0.5 * spacer
104. 3 :- 1* spacer
105. 4 :- 1.5* spacer
106. 5 :- 3* spacer

Here spacer:-16px

Width Prefixes in bootstrap:-

width:w-

Width Values:-

- w-25(25%)
- w-50(50%)
- w-75(75%)
- w-100(100%)

Shadow Prefixes in bootstrap:-

shadow-none

For Navbar:-

- Search navbar in the search bar of bootstrap → scroll down and copy that link and paste in your file inside the body tag and make changes on that

- Same like navbar you can get cards,forms,carousel,tables,…etc

To push website into github:-

- Create a repository in github
- And give repository name
- Post everything in a public

- Click on create repository
- After creating scroll down a little bit you can see their some commands like add, commit, branch..etc
- Open terminal (ctrl+/(backticks))

JavaScript(JS):-

Defination related to mern:- Js is a single threaded, synchronous programming language that converts static pages into the dynamic pages

Def:-JavaScript is a scripting language as well as a programming language

Introduced by

DAY-5 of Mern Full Stack

Git commands:-

107. git init
108. git config –global user.email "xxxxxx@gmail.com"
109. git config –global user.password "xxx"
110. git add
111. **to save all the changes:-** git commit –m "any msg"
112. **setting branch :-** go to repository and copy git branch code and paste in the terminal and click on enter
113. **git remote add origin**

JavaScript(JS):-

Defination related to mern :- Js is a single threaded, synchronous programming language that converts static pages into the dynamic pages

Def:-JavaScript is a scripting language as well as a programming language

114. Introduced by Brendan Eich
115. This is browser friendly
116. Js is used to make dynamic webPages
117. We have es6 modules those are known as ECMA Script
118. Scripting language not a compilation language

Single threaded:-It executes a single line and executes single task at a single time

119. If any error in the first line then it doesn't move to the second line

Synchronous:-

120. It haves some rules to execute, it execute one line by line, if any error gets it will exit from their and throws error
121. We can also apply asynchronous objects to synchronous by using await async objects and promises

Diff b/w scripting languages and programming languages:-

122. In scripting language we can't be converted into an executable code
123. In programming languages the code will be copiled and created to executable code

Two types of creating js:-

124. Internal :- body → bottom → script → code will be written
125. External :- we will keep external file link inside the head in href , external file must be saved with .js

Variables in js:-variables are containers to store data

126. **Using var** ex:- var a=10 we can redeclare and we can reassign
contains global scope

127. **Using let** ex:- let a=10 this can't be redeclared and we can be reassigned
contains block scope or functional scope

128. **Using const** :- ex:- const a=10 this can't be redeclared and can't be reassigned
contains block scope

129. **Without using anything** ex:- a=10

Hoisting :- it is a js behavior where the function and variable declarations are moved to the top of the scope that means they are moved to the global scope

Ex:- 1 console.log(a)

let a=10 we get reference error that means a is not initialized

Ex:-2 console.log(a)

const a=10 we get undefined

Block Scope / functional scope:-

130. The code inside the braces ({ })

Ex:- {

let a=20;

console.log(a)

}

Global scope:-

Ex:- let b=20

console.log(b)

Data Types in Js:-

Two types of data types

131. **Primitive data type**:- we can't store multiple values and multiple data types
132. String:- anything enclosed in the double quotes " "
133. Number
134. Boolean:- true or value
135. BigInt
136. Undefined :- not defining value for the variable
137. Null
138. Symbol
139. Object:- contains properties and methods

140. **Non primitive data type:-** we can store multiple values and multiple data types

- Object
- Arrays

Object Syn:-

let variable_name={

key:value,

key:value,

.

.

}

Operators in js:-

- Arithemetic operators :- +,-,*,%,/,++,--
- Comparison operators:- <,>,<=,>=,=== it also checks about type, checks strictly!=,!==,== it doesn't check about type, doesn't check strictly
- Bitwise operators :- & (bitwise and) ,|(bitwise or) ^(not) , ~(bitwise not) , <<(left shift),>>(right shift), >>>(right shift with zero)
- Logical operators:- &&(logical and), || (logical or) , ! (logical not)
- Assignment operators:- =,+=,-=,*=,/=,%=

Conditional Statements:-

- **if statement:-**

  syn:-

  ```
  if(expression / condition){

  }
  ```

- **else statement**

  syn:- :-

  ```
  if(expression / condition){
      //content to be evaluated if condition is true
  }else{
      //content to be evaluated if condition is false
  }
  ```

- **else-if statement:-**

  syn:-

  ```
  if(expression / condition){
      //content to be evaluated if condition1 is true
  }else if (condition2){
      //content to be evaluated if condition2 is true
  }else{

  }
  ```

- We will use prompt to take the output from the user
- **Ex:-**let input=prompt("enter input")

Switch Statement:- The switch statement in js is used to perform diff actions

- We have break and case in switch statements

Break

Case :- condition

Loops in JS:-

In js we have loops which offer a quick and easy way to do something repeatedly

While loop :- for single condition we use

Syn:- while(condition){

```
    //block of code to be executed

    }
```

For loop :- multiple conditions we can use this

Syn:-

```
for(expression1;expression2;expression3){

    //block of code to be executed

}
```

Generally loops have initialization :- where we start , condition :- is where to end and how to end ,iteration :- which helps condition to execute

for-of:-used for strings, to check every letter in the string

syn:-

```
for(let val of strVal){

    //do some work

}
```

for-in:- Used to get the keys in the objects

syn:-

```
for(let key in strVal){
        //do some work
}
```

Day:-6 Of Mern Full Stack

Practice Problems for for loops,operators:-

1.Ticket cost based on the age:-

```
let age=prompt("enter your age");
let isStudent=false;
if(age<12 && age>=0){
   console.log("Ticket Price: $5");
}else if(age>12 && age<=18 && isStudent===false){
   console.log("Ticket Price: $8");
}else if(age>=65){
   console.log("Ticket Price: $10");
}else{
   console.log("Ticket Price: $15");
}
```

2.Number Classifier:-

```
let number=prompt("Enter a number");
let result;
if(number===0){
   result="Zero";
}else if(number%2===0){
   result="Even";
   if(number>0){
```

```
            result+= " Positive";
        }else{
            result+=" Negative";
        }
    }else{
        result="Odd";
        if(number>0){
            result+=" Positive";
        }else{
            result+=" Negative";
        }
    }
```

3.Day of week checker:-

```
let number=prompt("Enter a Number in between 1-7");
switch (number) {
  case '1':
    console.log("Monday");
    break;

  case '2':
    console.log("Tuesday");
    break;
```

```
    case '3':
        console.log("Wednesday");
        break;
    case '4':
        console.log("Thursday");
        break;
    case '5':
        console.log("Friday");
        break;
    case '6':
        console.log("Saturday");
        break;
    case '7':
        console.log("Sunday");
        break;
    default:
    console.log("Invalid day number")
}
```

4.calculator

```
        let number1=prompt("Enter a number");
        let number2=prompt("Enter a number");
```

```
let operator=prompt("Enter an operator like +,-,*,/");


switch (operator){
   case '+':
      console.log(number1+number2);
      break;
   case '-':
      console.log(number1-number2);
      break;
   case '*':
      console.log(number1*number2);
      break;
   case '/':
      if(number2==0){
         console.log("Cannot divide by zero");
      }else{
         console.log(number1/number2);
      }
      break;
   default:
   console.log("Invalid operator");
}
```

5.leap year:-

```
let year=prompt("Enter a year");
if(year%4===0 && year%100===0){
   console.log("Leap Year")
}else{
   console.log("Not a Leap Year")
}
```

6.Vowel Checker:-

```
let alphabet=prompt("enter a alphabet");
if(alphabet==='a' || alphabet==="e" || alphabet==='i' || alphabet==='o' || alphabet==='u'){
   console.log("Vowel")
}else{
   console.log("Constant")
}
```

7.sum of digits:-

```
let number = 345;
let sum = 0;
while (number > 0) {
   sum += number % 10;
```

```
    number = Math.floor(number / 10);
}
console.log(sum);
```

8.factorial:-

```
let n=prompt("enter a number");
let fact=1;
for(let i=1;i<=n;i++){
    fact*=i
}
console.log(fact)
```

9.Reverse of a number:-

```
let n=prompt("enter a number/number");
let rev=""
for(let i=(n.length)-1;i>=0;i--){
    rev+=n[i]
}
console.log(rev)
```

10.palindrome of a string:-

```
let n=prompt("enter a number/number");
let input=n.toUpperCase()
let rev=""
```

```
for(let i=(input.length)-1;i>=0;i--){
   rev+=input[i]
}
if(input===rev){
   console.log("Palindrome")
}else{
   console.log("Not a Palindrome")
}
```


Array:-

141. It is a combination of similar data types and non similar data types

Examples:-

Let array=[“ironman”,”hulk”,”thor”,”batman”];

Let marks=[98,99,90,35,60]

Let info=[“rahul”, 86,”Delhi”]

Getting highest number in an array:-

For this we have multiple ways

142. Using sort function we will sort all the values in the array and we print last value from the array
143. Using max builtin function we can get max value easily
144. Using comparing

Ex:- way3

```
let array=[31,28,29,30];
let highnum=array[0];
for(let i=1;i<=array.length-1;i++){
```

```
        if(array[i]>highnum){
          highnum=array[i];
        }else{
          highnum=highnum;
        }
      }
      console.log(highnum);
```

Sum Of array:-

```
let array=[31,28,29,30];

let highnum=array[0];

for(let i=1;i<=array.length-1;i++){

  if(array[i]>highnum){

    highnum=array[i];

  }else{

    highnum=highnum;

  }

}
```

Array Methods:-

145. **Push**:- It adds value in the last of array
146. **Pop:-** delete from the end
147. **Concatenation:-** joins multiple arrays and return result
148. **Unshift:-**adds starting of array
149. **Shift:-** removes starting of value from the array
150. **Slice:**- returns a piece of array
    1. **Syn:**- slice(startIndex,endIndex)
151. **Splice:-** used to change the original array
    1. **Syn:-** splice(startIndex,deleteCount,newElement…)

Remove duplicates of array:-

```
let array=[1,1,2,3,4,4,5,5,6,6];

let uniqueArray=[];

for(let i=0;i<=array.length-1;i++){

   let isUnique=false;

   for(let j=0;j<=uniqueArray.length;j++){

      if(uniqueArray[j]===array[i]){

         isUnique=true;

         break;

      }

   }if(!isUnique){

      uniqueArray.push(array[i]);

   }

}

console.log(uniqueArray);
```


Objects:-

In Js objects is a data structures which can store the properties and methods

152. **Properties**:- A named value that describes the objects characteristics

153. **Methods:-** A functions that can be performed in a objects is called a methods

Syn:-

```
object{
        prop:value
}
```

Functions in Js:-

Block of code that performs a specific task, can be invoked whenever needed

154. We have 7 types of functions
155. **Named function:-** also called as pure function

```
Syn:- function named(){
        //block of code
      }
```

156. Anonymous function :-

```
Syn:- function(){
        //block of code
      }
```

157. **Function expression**:- it a function by getting the named function assigning to a variable

```
Syn:- let expressionFun =function(){
        //block of code
      }expressionFun()
```

158. Arrow function :-

```
Syn:- let arrowFun=()=>{
        //block of code
      }arrowFun()
```

159. **Immediate invoke function:-** we can call the function immediately in the starting

```
Syn:- function named(){
        //block of code
      }()
```

160. Callback function:-passing one function to the another function as an argument
161. **Higher order functions:-**a function can be passed as an argument to the another function more than one and it returns the output as a function

forEach Loop in Arrays:-Used to iterate and manipulate in the same array

162. We can use for the arrays and objects
163. arr.forEach(calBackFunction)

Syn:-arr.forEach(val){

```
        console.log(val);
}
```

map():-creates a new array and generates output in a new array

arr.map(callbackFun(value,index,array))

Syn:-

```
let newArr=arr.map((val)=>{
    return val*2;
})
```

Filter:-

Creates a new array of elements that give true for a condition

Eg:- all even numbers

```
letnewArray=arr.filter((val)=>{
    return val%2===0;
}
```

Reduce:-It performs some operations and reduces the array to a single value.it returns that single value

164. We have two values accumulator and current value

Syn:-

```
letreducearray=array1.reduce(accumulator,currentValue)=>
accumulator+currentValue,initalValue));
Console.log(reducearr);
```

Day:7 Of Mern Full Stack

DOM(Document Object Manipulation):-

165. It is a programme interface for html and xml documents.
166. It represents the structure of a documents as a tree of objects, allowing JavaScript to dynamically access and manipulate content, structure and style

Critical Rendering:-

The website works by using the following critical rendering steps

We have six types

167. DOM(document object model)
168. Css object model
169. Render tree (Js)
170. Combining of all those(dom,css,js)
171. Painting (displaying the website)
172. Composity:- after painting it will checks everything like what order it is, it will creates everything in a order way

Dom Manipulations:-

173. When dom manipulations are applied?

- When it goes to render tree then dom manipulations will be applied

174. How we can apply dom manipulations?

- By using selectors, event listeners, fetching we can apply dom manipulations

175. **Method ➔ Example➔ Returns**
176. **getElementById()** ➔ document.getElementById(“header”) ➔ Single Element
177. **getElementByClassName** ➔ document.getElementByClassName(“btn”) ➔Html Collection(live)
178. **getElementByTagName** ➔ document.getElementByTagName(“div”) ➔ HtmlCollection(live)
179. **querySelector()** ➔ document.querySelector(“.className”) ➔ first matching element
180. **querySelectorAll()** ➔ document.querySelectorAll(“button”) ➔

Example Using Id Selector:-

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 id="heading">Welcome to DOM</h1>
  <script>
    const heading=document.getElementById("heading");
    heading.textContent="DOM Manipulations script";
    heading.style.color="blue";
  </script>
</body>
</html>
```

Example using class selector:-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <button class="btn">Login</button>
  <script>
    const button=document.getElementsByClassName("btn");
    button[0].style.backgroundColor="blue"
    button[0].style.color="white"
    button[0].style.height="40px"
    button[0].style.width="80px"

  </script>
</body>
</html>
```

Example using type selector:-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
   <title>Document</title>
</head>
<body>
   <button>Login</button>
   <script>
      const button=document.getElementsByTagName("button");
      button[0].style.backgroundColor="blue"
      button[0].style.color="white"
      button[0].style.height="40px"
      button[0].style.width="80px"

   </script>
</body>
</html>
```

Example using querySelectorAll():-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
   <title>Document</title>
</head>
<body>
   <button class="btn">Login</button>
   <button class="btn">Submit</button>
   <script>
      const button=document.querySelectorAll(".btn");
         button.forEach(item=>{
```

```
            item.style.backgroundColor="blue",
            item.style.color="white",
            item.style.height="40px",
            item.style.width="80px";
    })
    </script>
</body>
</html>
```

Dom traversal properties:-

We have different types few are:-

firstChild:-we use this when we want to style only for first element

lastChild:- we use this when we want to style only for last element

nextSibling:-To apply styles for next element

previousSibling:- for previous element

Example:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <ul id="list">
        <li>Item1</li>
        <li>Item2</li>
        <li>Item3</li>
    </ul>
<script>
const list=document.querySelector("#list");
        const firstItem=list.firstElementChild;
        const lastItem=list.lastElementChild;
        firstItem.style.color="red";
        lastItem.style.color="blue";
```

```
        list.style.color="green";
        list.style.fontSize="25px";
    </script>
</body>
</html>
```

Changing content & styles dynamically:-

- To change only the text we can use textContent
- To change the text along with the tag we can use innerHtml

- **textContent** ➔ elment.textContent=”New Text” ➔ changes text
- **innerHtml** ➔ element.innerHtml=”Bold” ➔Changes html
- **createElement** ➔ document.createElement(div) ➔ used to create a element dynamically
- **appendChild** ➔ parent.appendChild(newElement) ➔ adds new element to the parent at the end
- **removeChild**➔ parent.removeChild(child) ➔ Removes a child

- **Adding Attributes directly:-**
  - **Style** ➔ element.style.color ➔ change css
  - **setAttribute()** ➔ element.setAttribute(“href”,”#”) ➔sets an attribute
  - **getAttribute()** ➔ element.getAttribute(“href”) ➔ gets an attribute

Example for inner html:-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
        <p id="maindiv">Hellooo</p>
          <script>
            const div=document.querySelector("#maindiv");
            div.innerHTML="<h1>this is added dynamically</h1>"
```

```
        div.style.color="blue";
        div.style.fontSize="15px"
            </script>

</body>
</html>
```

Example 1 for Adding attributes dynamically:-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
   <title>Document</title>
</head>
<body>
   <a id="anchortag">Link</a>
   <img id="imgtag">

   <script>
        const anchorTag = document.getElementById("anchortag");

        anchorTag.setAttribute("href", "https://www.youtube.com/");

        const link=document.querySelector("a");
        const href=link.getAttribute("href");
         link.setAttribute("target","_blank")
</script>
</body>
</html>
```

Example2:-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
 <a id="anchortag">Link</a>
<img id="imgtag">

    <script>
        const imgTag = document.getElementById("imgtag");
        imgTag.setAttribute("src", "./anochring.jpg");
        imgTag.setAttribute("alt","game image");
        imgTag.style.height="400px";
    </script>
</body>
</html>
```

Example for Remove Attribute:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <a id="anchortag">Link</a>
    <img id="imgtag">
    <script>
        const anchorTag = document.getElementById("anchortag");
        anchorTag.setAttribute("href", "https://www.youtube.com/");

        const link=document.querySelector("a");
        const href=link.getAttribute("href");
        link.setAttribute("target","_blank")
        link.removeAttribute("target");
    </script>
</body>         </html>
```

Event Handling:-

Example:-

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=<device-width>, initial-scale=1.0">
   <title>Document</title>
</head>
<body>
   <button id="btn">Submit</button>
   <script>
      const button=document.getElementById("btn");
      btn.addEventListener("click",()=>{
         alert("Form is Submitted");
      })
   </script>
</body>
</html>
```


- When we click on the button then we get an alert box with form is submitted text
- In the above example we are using onClick event listener

Example for getting output in console:-

```
btn.onClick=()=>{
        console.log("Button is Clicked");
}
```

Form Handling:-

- Used to prevent the form submissions
- We use preventDefault it is used when we don't want to gt the value when the site is refreshed
- We use value to get the value of the input ,

Example for form submission:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=<device-width>, initial-
scale=1.0">
    <title>Document</title>
</head>
<body>
    <form id="loginForm">
        <input type="text" id="username" placeholder="Username"/>
        <button type="submit">Submit</button>
        <p id="value"></p>
    </form>
    <script>
        const input=document.getElementById("username");
        const form= document.getElementById("loginForm");
        form.addEventListener("submit",(e)=>{
            e.preventDefault();
            const inputValue=input.value
            console.log(inputValue)
        })

    </script>
</body>
</html>
```

DAY-9 OF Mern Full Stack

SetTimeout():- used to delay for sometime takes in milliseconds
1000milliseconds=1second

EX:-

```
console.log("Start");
setTimeout(function(){
    console.log("Callback after 2 seconds");
},2000);
console.log("End");
```

O/P:-

Start

End

Callback after 2 seconds

Callback Hell :- very difficult to understand

181. This is the drawback of callbacks
182. We have lot of nested callbacks
183. **How to avoid:-**
    1. By using named functions to flatten structure, promises, Async/Await

Example:-

```
console.log("Start");
setTimeout(()=>{
    console.log("1st step done");
    setTimeout(()=>{
        console.log("2nd step done");
```

```
        setTimeout(()=>{
            console.log("3rd step done");
            setTimeout(()=>{
                console.log("4th step done");
            },1000);
        },1000);
    },1000);
},1000);
```

o/p:-

Start

(after 1s) 1st step done

(after 2s) 2nd step done

(after 3s) 3rd step done

(after 4s) 4th step done

Promises:-

It is a javascript object that represent eventual completion (or failure) of an asynchronous operation.

184. To get the asynchronous programming we will use promises
185. If we write synchronous programme it will also possible,we can also use promises for synchronous
186. It contains three states:-
    1. Pending- Initial state, neither fulfilled nor rejected
    2. Fulfilled – operation completed successfully
    3. Rejected - operation failed

Syn:-

```
const promise =new promise((resolve,reject)=>{
        if(success){
            resolve(“Success message”);
        }else{
                reject(“Error Message”)
        }
});
```

Ex:-1To check age using promises in synchronous operations:-

```
        const checkAge=new Promise((resolve,reject)=>{
           const age=20;
           if(age>=18){
              resolve("Access Granted");
           }else{
              reject("Access Denied");
           }
        });
        checkAge
           .then((message)=>{
              console.log(message);
           })
           .catch((error)=>{
              console.log(error);                    });
```

O/P:- Access Granted

****Important example for promises is the below one**

EX:-2 asynchoronous operations in promises

```
function fetchData(){
   return new Promise((resolve,reject)=>{
      setTimeout(()=>{
         const success=true;
         if(success){
            resolve("Data fetched successfully");
         }else{
            reject("Failed to fetched data");
         }
      });
   });
}

fetchData()
   .then((data)=>console.log(data))
   .catch((err)=>console.log(err));
```

O/P:-

Data fetched successfully

Promises Chaining:-drawback of promises

187. The promises chaining disadvantage was handled by async/await
188. Better than nested callbacks
189. Easier error handling
190. To give one second for all promises we use promise.all
191. Disadvantage of promise.all if any one will be rejected we will doesn't get output
192. Advantage if all promises will be resolved then we get the output all at a time

Syn:-
```
function step1(){
    return new promise((resolve=>{
        setTimeout(()=>{
            console.log("Step1 done");
            resolve();
        },1000);
    });
}

function step2(){
    return new promise((resolve=>{
        setTimeout(()=>{
            console.log("Step2 done");
            resolve();
        },1000);
    });
}
```

step1()

.then => step1()

.then(()=> console.log(“all steps completed”));

Async and Await:-

These are modern javascript features used to handle asynchronous operations in a clean, readable, way-especially when working with promises

Async:-

193. The async keyword is added to a function to make it return a promise automatically
194. Even if a function returns a value, Javascript wraps it as a promise

Example:-

```
async function greet(){
        return “Hello”
}
greet().then(msg=> console.log(msg));
```

O/P:- Hello (this output is synchronous when we use await it wil changes to async)

Await:-

195. It is a keyword, it can only used inside the async
196. It pauses the execution of the async function until the promise is resolved, then it resumes with the result.

Example:-

```
async function getData(){
        console.log(“Start Fetching….”);
        const result=await fetchData();
        console.log(result);
        console.log(“Done”);
}
```

```
getData();
```

DAY 10 Of Mern Full Stack

MAP:- The map() method creates a new array by applying a function to each element of the original array.

Ex:- const numbers=[1,2,3,4];

const doubled=numbers.map(num=>num*2);

console.log(doubled)

o/p:- [2,4,6,8]

Filter:- The filter() method creates a new array with only the elements that pass a test (return true)

Ex:- const numbers=[1,2,3,4,5]

const even=numbers.filter(num=> num%2===0);

console.log(even);

o/p:- [2,4]

Reduce:- The reduce() method reduces the array to a single value by applying a function to each element and accumulating the result

197. **Accumulator :-** it is the value we are taking, it will same for all iterations
198. **Current :-** the current iteration value

Ex:- const numbers=[1,2,3,4]

const sum=numbers.reduce((accumulator,current)=> accumulator+current,0);

console.log(sum);

o/p:- 10

Spread:-The spread operator is used to unpack elements from arrays or objects

199. We can easily merge or copy two arrays or objects
200. Copy arrays

201. Passing array elements as individual arguments
202. If we change original array then original array we will be also changed to updated array

Ex:- 1 Copying an array

```
const arr1=[1,2,3];
const arr2=[…arr1];
```

o/p:- [1,2,3]

Ex:-2 Merging arrays

```
const arr1=[1,2,3];
const arr2=[…arr1];
const arr3=[4,5];
const merged=[…arr1,…arr3];
```

o/p:- [1,2,3,4,5]

Ex:-3 Passing elements to a function

```
function  add(a,b,c){
        return a+b+c;
}
const numbers=[1,2,3];
consle.log(add(…numbers));
```

o/p:-6

Ex:-4 Copying and modifying objects

```
const user ={name:"Alice",age:25}
const updatedUser={…user,age:26};
console.log(updatedUser);
```

o/p:- {name:"Alice",age:26}

Rest Operator:- The rest parameter is used to collect multiple values into an single array

203. It destructures the objects or arrays
204. If we want a particular part of an array or object we can use destructuring in rest

Ex:-1 Function Parameters

```
function sum(…numbers){
```

```
        return numbers.reduce((total,num)=> total+num,0);          }
    console.log(sum(1,2,3,4));
```

o/p:- 10

Ex:-2 Array destructuring

```
    const [first,…rest]=[10,20,30,40];
    //first=10        rest=[20,30,40]
```

Ex:-3 Object destructuring

```
    const person={name:”Bob”,age:30,city:”Paris”};
    const {name, …otherDetails}=person;
    //name=”Bob”          otherDetails={age:30,city:”Paris”}
```

Shallow Copy:-

A Shallow copy copies only the top-level values. If the object has nested objects or arrays, the nested references are still shared

205. If we apply changes in one thing the changes are applied to another one also

Ex:-

```
const original={
    name:”Alice”,
    details:{
        age:25
    }
};
const shallowCopy={…original};
shallowCopy.details.age=30;
console.log(original.details.age);
```

o/p:-30

Deep Copy:-A deep copy copies all levels of the object or array. No references are shared- everything is fully cloned

206. structuredClone() is a builtin function, we will share object inside that
207. here original array is not affected

Ex:- const original={

name: “Alice”,

details:{
age:25

}

};

const deepCopy=structuredClone(original);

deepCopy.details.age=30;

console.log(original.details.age)

o/p:- 25

JSON:- JavaScript Object Notation

208. It is a lightweight format to store and exchange data. It’s like a javascript object but I string format.
209. In javascript , json is treated as a string and you must parse it to use it like a normal object
210. Json data is in a string format.

Parse Json String to Object:-

const jsonString='{"name":"Alice","age":25}';

const obj=JSON.parse(jsonString);

console.log(typeof(jsonString));

console.log(obj.name);

console.log(typeof(obj));

o/p:-string

Alice

Object

Convert object to json string:-

const jsonString=JSON.stringify(jsonObject)

console.log(jsonString)

console.log(typeof(jsonString))

o/p:- {“name”:”Alice”,”age”:25}

string

Fetch:-

211. Jsonplaceholder to get free api’s open in a browser ➔scroll down upto resources➔posts➔copy url and paste inside the fetch()

Example for fetching extenal link:-

```
fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(res=>res.json())
    .then(user=>{
      console.log("Name:",user.name);
      console.log("Email:",user.email);
    });
```

MiniProject on Js:-Weather App

To get weather app api:-

212. open➔open weather map➔signin➔click on your username➔click on my api keys➔give api key name ➔ click on generate you will get an api
213. keep that api key inside the js in a variable api key
214. now click on api in that page➔scroll down you found daily forecast 16 days➔click on apidoc➔scroll down and copy api call to gee api url

For Movies App:-

215. We can use tmdb and➔ signup➔login
216. We want to store data into database to store we have local storage and session storage
217. If we are storing in local storage then if we close our browser unexpectedly then the data will be saved and after opening next time we can continue our work,the previous data will not be lose
218. When we come out from session storage we will lose the session data so we use local storage mostly
219. In tmdb we use DOMContentLoaded event in JavaScript to make sure the html content of a web page has been completely loaded and parsed before we try to run our scripts that interact with the DOM (Document Object Model)
220. Why it is important? :- If your JavaScript tries to access elements (like buttons, inputs, or divs) before they are fully loaded, it might result in errors like null or undefined because those elements don't exist in the DOM yet

DAY:14 of Mern Full Stack

React:-

React is created for single page application mainly built for reusable complexity and component based structure

It is a JavaScript library used to build reusable UI components and single page applications (SPAs) with fast rendering using the virtual DOM

221. Due to the virtual DOM we don't have lazy loading, reduces refreshing rate and manipulate and it improves performance
222. Before clicking live server the react will create and load virtual DOM , whenever we are clicking live server it will moves to the original DOM
223. Virtual rendering is present in the browser itself
224. React itself contains reloaded speed and refreshing

History of react:-

225. Created by Jordan Walke, he is a software engineer at facebook
226. Developed in 2011
227. And later by using that Instagram is created in 2012
228. React was open sourced in 2013 in may by facebook

Why React?

229. Facebook faced updating the DOM efficiently, managing complex UIs with a lot of dynamic data
230. They needed a fast reusable and maintainable solution, react solved with this
231. React is a component based structure
232. One way data flow (single page),from top to bottom the data flows, this makes debugging eaiser
233. It handles the rendering efficiently
234. It contains large developer community and third-party libraries

Vite is the traditional method to create react application, it contains updated modules

Whatsapp, Instagram, Netflix are developed using React

Step-by-Step Installation for React:-

235. First we need to install node Js ➔ type nodejs download➔nodejs.org/en/download➔click on download➔ after downloaded ➔double click on it ➔ the latest version is 22➔after downloaded click all next and click install
236. For checking correctly installed or not in cmd type node or node –version
237. Create new folder➔reactapp (folder name) ➔file explorer➔cmd➔code . ➔ open in vsc➔ open terminal
238. npm create vite@latest . ➔if you get errors ➔ type Get-ExecutionPolicy and Set ExecutionPolicy RemoteSigned in windows powershell after this if you get remote signed then completed setting
239. it asks to select framework ➔ click on react and click enter
240. asks to select variant ➔ select javascript and click on enter
241. type ➔ npm I ➔we get found 0 vulnerabilities then successfully installed
242. npm run dev ➔ open local link you will get default code and output

Default packages and code are like as below:-

Default Output:-default counter app code will be there

React SetUp was completed successfully we want to do every time when we create a new project

243. We have extension jsx:- javascript extension
244. Here we can include html code directly
245. Combination html+ js =jsx
246. We need to create everything with .jsx extension when we are using vite

Folder Structure:-

247. We can node modules in this we have lot of packages, installed automatically when we type npm I command
248. Next we have public all common files will be placed
249. The code will be written in src
250. Here we have only one file index.html it is the main html file
251. main.jsx is the main javascript file
252. in main.jsx file we have a tag StrictMode by using this the errors will be created, we will get errors clearly and perfectly
253. what ever we change in app.jsx that will be reflect on main.jsx
254. what ever we change in main.jsx those are reflected in index.html
255. After creating app everytime remove all the code present in the index.css due to this styles will not be override
256. package-lock.json the starting modules what we will be installed

257. package.json it contains scripts and the packages we are going to install further, to check the package installed or not or we can check here if we want version we can check here
258. we have readme.md there is no need of this you can delete this

Functional Component:-

A functional component is a JavaScript function that returns JSX

Syn:-

```
function Welcome(){
    return <h2> Welcome to React </h2>
}
```

(OR arrow function)

Syn:-

```
const Welcome=()=> <h2> Welcome to React </h2>;
```

To Create Functional Component :-

We will type rafce (react functional component exporting) we will get the default code, for this we want to install the extension es7 ➔ ES7+ React/Redux/React-Native snippets

Props in React:-

Allows you to pass the data from a parent component to a child component

259. Used only for passing simple data if the complexity is increased we use context api
260. Context api is for medium projects only, even the complexity will increases then we use redex

Example:-

```
const User=(props)=>{
    return <h3>Helloo, {props.name}</h3>
}
```

```
function App(){
        return <User name=”Alice”/>;

}
```

In parent component what data we want to pass that will be written,and that is passed to child as props

261. While importing anything in react don’t import that manually in the below inside return type your component name what you want to import than than click on enter that will be imported above automatically

Example for passing single props:-

App.jsx:-

```
import React from 'react'
import User from './components/User'
import './App.css'
const App = () => {
 return (
  <>
   <User name="Prudula"/>
  </>
 )
}

export default App
```

User.jsx:-

```
import React from 'react'

const User = (props) => {
 return (
  <div><h1>This is User Component {props.name}</h1></div>
 )
}
```

```
export default User
```

Props Drilling:-

It means passing the data from a top-level component down to deeply nested components even if intermediate components don't need to use that data,but just passing it along

262. Props are passed from one component to another component that do not need the data but only help in passing it through the tree that is called as **"prop drilling"**

263. In the above example the featuresSection doesn't need the data but only for passing that data to childs components we will pass through that one
264. As the application will increase the passing no.of propos will be increased and unnecessary data will be passed
265. To pass the data only to the childrens without passing them to all components we have a react context which is to be discussed in further classes

Example for Props Driling :-

Passing Props from App.jsx and to User.jsx and to Profile.jsx:-

App.jsx:-

```
import React from 'react'
import User from './components/User'
import './App.css'
const App = () => {
 return (
  <>
   <User name="Prudula"/>
  </>
```

```
 )
}
export default App
```

User.jsx:-

```
import React from 'react'
import Profile from './Profile'

const User = (props) => {
 return (
  <div>
    <h1>This is User Component</h1>
    <Profile profileName={props.name}/>
  </div>
 )
}

export default User
```

Profile.jsx:-

```
import React from 'react'
const Profile = (props) => {
 return (
  <div>
    <h1>This is Profile Component {props.profileName} </h1>
  </div>
 )
}

export default Profile
```

Styling in React:-

- **Inline Styling:-**

```
const style={
        color=”blue”,
        backgroundColor:”lightgray”
}
return <h1 style={style}>Styled Heading</h1>;
```

2. CSS styling:-

In App.css:-

```
.title{
        font-size:24px;
}
```

Day:-15 & Day:- 16 Of Mern Stack

React Hooks

266. Hooks are special JavaScript functions
267. Introduced in React 16.8 version
268. That allow functional components to use features like state, lifecycle methods, and more features that were earlier available only in class components
269. Totally we have 7 types of react hooks
270. Mostly important hook is useState and useEffect
271. **UseState:-** used for manipulating the state of the application
    1. It is a react hook that allows local state management in a functional component.
    2. **Syn:-** const [state, setState]=useState(initialValue);
        1. **state:**- current value
        2. **setState:-** function to update the previous value
        3. **initialValue :-** starting value for the state (string,number,array,object,….etc)
272. **UseEffect :-** used for side effects and lifecycle methods
    1. Side effects means interacting with the outside world
    2. It is a hook that lets you run side effects like fetching data, setting up subscriptions, or manually updating the DOM or for toogling
    3. We will do all dom manipulations inside the useEffect
    4. Dependencies ➔ while rendering it will check those dependencies and apply those effects in the website
    5. **Examples :-**
        1. API Calls
        2. Dom updates (scroll, title,..etc)
        3. Subscriptions
        4. Timers
273. **UseRef:-** Accessing and persisting DOM values
274. **useReduce:-** we will use this in redux
275. **useMemo:-** for calculation parts we will use this,for rerendering we use this

276. **useCallback:-** for rewriting we use this
277. **useContext**

UseState Example:-

FormInput.jsx:-

```
import React, { useState } from 'react'

const FormInput = () => {
   const [name,setName]=useState("");
   const [email,setEmail]=useState("");

 return (
  <div style={{marginLeft:"800px"}}>
     <form action="submit">
        <label htmlFor="name">Name:</label>
        <input style={{height:"35px",marginBottom:"10px",fontSize:'25px'}}
 type='text' onChange={e=> setName(e.target.value)}/><br/><br/>

        <label htmlFor='email'>Email: </label>
        <input style={{height:"35px",marginBottom:"10px",fontSize:'25px'}}
type='email' onChange={(e)=> setEmail(e.target.value)}/><br/><br/>
        <button
style={{backgroundColor:'blue',marginLeft:'100px'}}>Submit</button>
     </form>
     <div
style={{backgroundColor:"blue",padding:'10px',marginTop:"40px",height:'2
00px',paddingTop:"90px",width:"300px",borderRadius:"20px"}}>
        <p style={{fontSize:'20px'}}>Name: {name}</p>
        <p style={{fontSize:'20px'}}>Email: {email}</p>
     </div>
  </div>
 )
}
```

```
export default FormInput
```

App.jsx:-

```
import React from 'react'

const App = () => {

 return (
  <>
   <FormInput/>
  </>
 )
}

export default App
```

Day:-17 of Mern Stack

Custom Hooks:-

It is a Js function whose name starts with “use” and that may call other hooks inside it.

Why use it:-

278. Extract and reuse logic across components
279. Keep components clean and dry

How to create:-

import { useState } from ‘react’

React Hooks:-

280. Used for dynamic
281. To manipulate state manipulations we use this

Types of hooks:-

282. **useState:-** used to change the previous state, local access used to access over a single component
283. **useRef:-** It allows you to target the dom elements, if we want to keep focus on only one particular item
284. **useEffect:-**used for fetching and loading the data
285. **useMemo:-**it memorizes the previous data and manage the next one
286. **useContext:-**global access, used to access over a multiple components
287. **useReducer:**-we use this for something like cart with multiple actions we can use this
288. **useCallback:**-

Example:-

```
const [liked,setLiked]=useState(false);

return(
```

```
        <button onClick={()=>setLiked(!liked)}></button>
);
```

Day:-17 of Mern Stack

Custom Hooks:-

It is a Js function whose name starts with “use” and that may call other hooks inside it.

Why use it:-

289. Extract and reuse logic across components
290. Keep components clean and dry

How to create:-

import { useState } from ‘react’

React Routing:-

First we want to install the react router for the type following commands in terminal

DAY:-18 of Mern Stack

WhenEver you start creating new app :-

291. npm create vite@latest . ➔after this command
292. npm i

If you want routing:-

293. npm i react-router-dom

want to use bootstrap:-

294. npm i react-bootstrap

want to run your app:-

295. npm run dev

To Use bootstrap in react:-

296. For normal bootstrap➔ Go to bootstrap5➔in the bundle➔ we have script tag➔ copy that and paste in ➔your folder in the index.html➔inside body tag➔at the end
297. In the terminal type command like npm I react-bootstrap
298. To check whether installed correctly or not go to the package.json and check inside the dependencies
299. For react bootstrap➔type react bootstrap in your browser➔ type navbar components➔choose what type of navbar you want➔copy that code and paste in your

navbar component➔and keep that component in the app.js file.

300. `import 'bootstrap/dist/css/bootstrap.min.css';`
301. Import the above line in main.jsx

UseNavigate:-

To navigate for a particular component we can use this without giving link
Here the below is the **basic example**

NavbarComponent.jsx:-

```
import Button from 'react-bootstrap/Button';
import Container from 'react-bootstrap/Container';
import Form from 'react-bootstrap/Form';
import Nav from 'react-bootstrap/Nav';
import Navbar from 'react-bootstrap/Navbar';
import NavDropdown from 'react-bootstrap/NavDropdown';
import { useNavigate } from 'react-router-dom';

function NavScrollExample() {
 const navigate=useNavigate()

 const handleAbout=()=>{
```

```
  navigate('/about')
 }

 return (
  <Navbar expand="lg" className="bg-body-tertiary">
   <Container fluid>
    <Navbar.Brand href="#">Navbar scroll</Navbar.Brand>
    <Navbar.Toggle aria-controls="navbarScroll" />
    <Navbar.Collapse id="navbarScroll">
     <Nav
      className="me-auto my-2 my-lg-0"
      style={{ maxHeight: '100px' }}
      navbarScroll
     >
      <Nav.Link href="/Home">Home</Nav.Link>
      <button onClick={handleAbout}>About</button>
      <Nav.Link href="/contact">Contact</Nav.Link>
      <Nav.Link href="/skills">Projects</Nav.Link>
      <Nav.Link href="/projects">Skills</Nav.Link>

     </Nav>
     <Form className="d-flex">
      <Form.Control
       type="search"
       placeholder="Search"
       className="me-2"
       aria-label="Search"
      />
      <Button variant="outline-success">Search</Button>
     </Form>
    </Navbar.Collapse>
   </Container>
  </Navbar>
 );
}
```

```
export default NavScrollExample;
```

App.jsx:-

```
import React from 'react'
import {BrowserRouter as Router,Routes,Route} from 'react-router-dom'
import Home from './componets/Home'
import NavScrollExample from './componets/NabarComp'
import About from './componets/About'
import Skills from './componets/Skills'
import Projects from './componets/Projects'
import Contact from './componets/Contact'

const App = () => {
 return (
  <>
   <Router>
    <NavScrollExample />
    <Routes>
     <Route path='/home' element={<Home/>} />
     <Route path='/about' element={<About/>}/>
     <Route path='/skills' element={<Skills/>}/>
     <Route path='/projects' element={<Projects/>}/>
     <Route path='/contact' element={<Contact/>}/>

    </Routes>
   </Router>
  </>
 )
}

export default App
```


Types of Routes:-

- **Static Routing:-** we want import routes,router,BrowserRouter
  - Inside browserRoute we will write router inside router we write route inside route we have path and element

- **Dynamic Routing:-** getting data from any api's for negotiating the static routing things this was introuduced
- Inside the path we want to include the : after that symbol we can write anything like userId or categories
- We want use useParams
- **UseNavigate:-**
- We define navigate function and we store that function in any variable after storing when we access we can get that
- **Nested Routing:-**
    - If we want to place inside route another routes, keeping single route in that we wrap another two routes
    - We want to use outlet
- **Protected Routing:-**
    - Simply to use conditional rendering we can use this
    - We will pass parameter as a children
    - For example here we are taking authentication simply true if it is true navigate to the dashboard, when it is false it navigate to the login
    - We wrap that route inside the protected route

For dynamic routing:-

For dynamic routing inside route path we will take the path by keeping : like as below
<Route path="/user/:userId" element={<UserProfile/>}/>

302. Inside useState we will give null when developer want to say nothing is there in that,if we can't give anything by default it will take undefined
303. In dynamic routing while passing we will pass by using useParams

Small example using useEffect, UseState and routers:-

UserProfile.jsx:-

```
import React, { useEffect, useState } from 'react'
import { useParams } from 'react-router-dom'

function UserProfile(){
  const {userId}=useParams();
  const [user,setUser]=useState(null);

  useEffect(()=>{
```

```
      fetch(`https://jsonplaceholder.typicode.com/users/${userId}`)
      .then(res=>res.json())
      .then(data=>setUser(data));
    },[userId]);

    if(! user) return <div>Loading....</div>
    return(
      <div>
        <h1>Name: {user.name}</h1>
        <p>Email: {user.email}</p>
      </div>
    )

  }
  export default UserProfile
```

App.jsx:-

We will add route UserProfile as below

```
import React from 'react'
import {BrowserRouter as Router,Routes,Route} from 'react-router-dom'
import Home from './componets/Home'
import NavScrollExample from './componets/NabarComp'
import About from './componets/About'
import Skills from './componets/Skills'
import Projects from './componets/Projects'
import Contact from './componets/Contact'
import UserProfile from './componets/UserProfile'

const App = () => {
 return (
  <>
   <Router>
    <NavScrollExample />
    <Routes>
     <Route path='/home' element={<Home/>} />
     <Route path='/about' element={<About/>}/>
     <Route path='/skills' element={<Skills/>}/>
     <Route path='/projects' element={<Projects/>}/>
     <Route path='/contact' element={<Contact/>}/>
     <Route path="/user/:userId" element={<UserProfile/>}/>
```

```
      </Routes>
    </Router>
   </>
  )
}

export default App
```

Output:-

For output in url search like http://localhost:5173/user/1

If you want all the users keep that in inside the forEach()

Nested Routes:-

304. One route inside another route or two or more routes inside the main route
305. We will have outlet here, outlet renders nested routes
306. All the routings related are inside the react-router-dom

Small Example is here below:-

DashboardLayout.jsx:-

```
import React from 'react'
import { Link, Outlet } from 'react-router-dom'

const DashboardLayout = () => {
 return (
  <div>
   <h2>Dashboard</h2>
   <nav>
    <Link to="profile">Profile</Link>
    <Link to="settings">Settings</Link>
   </nav>
   <Outlet/> {/* it is used to renderr the nested routes here */}
  </div>
 )
}

export default DashboardLayout
```

App.jsx:-

```
import React from 'react'
import {BrowserRouter as Router,Routes,Route} from 'react-router-dom'
import DashboardLayout from './componets/DashboardLayout'
import Profile from './componets/profile'
import Settings from './componets/settings'
const App = () => {
 return (
  <>
   <Router>
        <Routes>
                //nested route
      <Route path='dashboard' element={<DashboardLayout/>}>
       <Route path='profile' element={<Profile/>}/>
       <Route path='settings' element={<Settings/>}/>
      </Route>
     </Routes>
   </Router>
  </>
 )
}
export default App
```

Protected Routes:-

- Nothing but conditional rendering
- If login we will move to one else we will move to other one

ProtectedRoute.jsx:-

```
import { Navigate } from "react-router-dom";

export default function ProtectedRoute({children}){
   const isAuthenticated=false;
   return isAuthenticated?children:<Navigate to="/profile" replace/>;
}
```

App.jsx:-

```
<Route path='/admin' element={
            <ProtectedRoute>
              <DashboardLayout/>
            </ProtectedRoute>
```

```
}/>
```

We will write the above route in app.jsx

LazyLoading:-

307. In this we will keep the loading when that was loading
308. We have lazy and suspense here
309. Inside suspense we can write out routes
310. Lazy dynamically imports components dynamically
311. Suspense is for getting the callback things

Example:-

```
import { lazy, Suspense } from 'react';

Const Home =lazy(()=> import('./Home'));

function App(){

return (
    <Suspense fallback={<div>Loading…</div>}>

        <Routes>

            <Route path="/" element={<Home />} />

        <Routes/>

</Suspense>

);

}
```

DAY:-19 Of Mern Stack

What is the context API:-

The context API is a built in feature in react that allows you to share data across the component tree without having to pass props manually at every level (props drilling)

Why do we need context api?

To avoid prop drilling and when a value needs to passed from a parent component to deeply nested children, we end up passing props through many intermediate components that don't even use the data themselves.

Ex:-

```
<GrandParent>
        <Parent>
                <Child>
                        <GrandChild data={value}/>
                </Child>
        </Parent>
<GrandParent>
```


Structure of Context API:-

312. Create context
313. Provide context
314. Consume context

Create Context:-

```
export const ThemeContext=createContext();
```

315. This returns an object with provider & consumer
316. By using above line we can creates a context object

ThemeContext.jsx:-

```
import { createContext } from "react";
export const ThemeContext=createContext();
```

Provide Context:-

ThemeProvider.jsx:-

```
import { useState } from "react"
import { ThemeContext } from "./ThemeContext"

const ThemeProvider=({children})=>{
  const [theme,setTheme]=useState('dark')

  return(
    <ThemeContext.Provider value={{theme,setTheme}}>
      {children}
    </ThemeContext.Provider>
  )
}
export default ThemeProvider
```

ToggleComponent.jsx:-

```
import { useContext } from "react"
import { ThemeContext } from "./ThemeContext"

const ToggleComponent=()=>{
  const { theme, setTheme }=useContext(ThemeContext);
  console.log("Current Theme:", theme);

  return(
    <div>
      <h1 style={{ color: 'white' ,backgroundColor:"blue"}}>Theme is {theme}</h1>
      <button onClick={()=>setTheme(theme=="dark"?"light":"dark")}>Toggle Theme</button>
    </div>
  )
}
export default ToggleComponent
```

App.jsx:-

```
import React from 'react'
import ToggleComponent from './ContextApi/ToggleComponent'
const App = () => {
 return (
          <>
   <ToggleComponent/>
```

```
    </>
  )
}

export default App
```

Main.jsx:-

```
import { StrictMode } from 'react'
import 'bootstrap/dist/css/bootstrap.min.css';
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import ThemeProvider from './ContextApi/ThemeProvider.jsx';


createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ThemeProvider>
      <App/>
    </ThemeProvider>
  </StrictMode>,
)
```


Where we use context api:-

317. For authentication
318. Theme toggling
319. Language settings
320. Cart data in small e-commerce apps
321. Global modals, alerts…etc
322. Mostly use for medium applications
323. For small applications we can use props
324. For large we can use redux

Where we avoid context api:-

325. High Frequency state updates(like animations,timers)
326. Large-Scale state management (consider Redux for complex apps)

Advantages of context API:-

327. No need for prop drilling
328. Simple and easy to implement
329. Good for small to medium global scopes

Limitations of context API:-

330. Not ideal for high-frequency updates (can cause re-renders)
331. Not Observable like Redux (no dev tools or time-travel debugging)
332. Can't persist state out of the box
333. Can become hard to manage in large apps if overused

Few Interview Questions of context API:-

334. What is context api?
335. What problems does context api solve?
336. What is context API difference from redux?
337. How do you avoid unnecessary re-renders with Context?

Controlled Components:-

338. React has full control over input using state
339. Instead of taking two separate useStates we can take like as below by keeping all in a single object

const [form,setForm]=useState({email:'',password:''});

Example for Controlled Form:-

```
import React, { useState } from 'react'

const Form = () => {
  const [form,setForm]=useState({email:'',password:''});
  const [formData,setFormData]=useState(null)

  const handleChange=(e)=>{
   setForm({...form,[e.target.name]:e.target.value});
  };

  const handleSubmit=(e)=>{
   e.preventDefault();
   setFormData(form)
   setForm('')
  }


 return (
  <>
   <div>
     <form>
       <label>Email:- </label>
```

```
            <input name="email" type='email' placeholder='Enter Your Email' value={form.email}
onChange={handleChange}/><br/>
            <label>Password:- </label>
            <input name="password" type='password' placeholder='Enter Your Password'
value={form.password} onChange={handleChange}/><br/>
            <button onClick={handleSubmit}>Submit</button>
          </form>
      </div>
     {formData && (
      <div>
        <p>{formData.email}</p>
        <p>{formData.password}</p>
      </div>
     )}


    </>
  )
}

export default Form
```

Example for UnControlledForm:-

```
import React, { useRef } from 'react'

const UnControlledForm = () => {
    const inputref=useRef();

    const handleSubmit=(e)=>{
        e.preventDefault();
        alert(`Value: ${inputref.current.value}`);
    }
  return (
    <form onSubmit={handleSubmit}>
        <input ref={inputref} type='text' placeholder='Enter Name'/>
        <button type='submit'>Submit</button>
    </form>
  )
}

export default UnControlledForm
```

FormValidations Example:-

```
import React, { useState } from 'react'

const FormValidations = () => {
```

```
  const [email,setEmail]=useState('')
  const [password,setPassword]=useState('')
  const [error,setError]=useState('')

  const validate=()=>{
    if(!email.includes('@')){
      setError("Invalid email address");
      return false
    }else if(password.length<8){
        setError("Password must be min 8 characters");

        return false

    }else if(!/\d/.test(password)){
      setError("Include at least one number in password")
      return false
    }else if(!/[a-z]/.test(password)){
      setError("Include at least one lowercase alphabet")
      return false
    }else if(!/[A-Z]/.test(password)){
      setError("Include at least one uppercase alphabet")
      return false
    }
    setError('');
    return true
  }

  const handleSubmit=(e)=>{
    e.preventDefault();
    if(validate()){
      alert("Submitted")
    }

  }

 return (
  <form onSubmit={handleSubmit}>
    <label>Email: </label>
    <input value={email} onChange={(e)=>setEmail(e.target.value)}
      placeholder='Enter Your Email'/><br/><br/>
    <label>Password</label>
    <input type='password' value={password} onChange={(e)=>setPassword(e.target.value)}
      placeholder='Enter Your Password'/>  <br/><br/>
    <button type='submit'>Submit</button>
    {error && <p>{error}</p>}
  </form>
 )
}

export default FormValidations
```

Day-20/36 of Mern Stack

Don’t ask in interviews for freshers

Create new folder for this application

While creating new folder make sure to give the commands

340. npm create vite@latest .
341. npm i

Redux:-

342. To manage state larger we will use this one
343. Redux is a state management library for javascript apps (mostly used with react) that helps you manage and centralize the application state.
344. Single source of truth- One centralized place to store the state of your entire application
345. To negotiate props drilling

Why Redux ?:-

Problem:-

346. As apps grow, state becomes harder to manage
347. Props drilling becomes messy
348. Components dispatch actions to update state

How data flow in redux:-

Renders from UI ➔dispatch(action) ➔reducer ➔ store➔ UI re-renders

Step-by-Step Redux with example:-

Step1:- Install redux

npm install @reduxjs/toolkit react-redux

349. if you want to check whether installed successfully or not check in package.json

Step:-2 Create a slice:-

Create counterslice.jsx I created this inside the slices component

350. While creating slice we need to give first name ➔ initialState➔then reducers ➔inside reducers we will write our logic
351. We are going to store that slice in store.jsx
352. Store.jsx is like a provider
353. And we will wrap that store.jsx in main.jsx and then we use that in the components here I have Counter.jsx component
354. To create another reducer we have extra reducers when logic will be more
355. Taking a variable called counterSlice we are going to create slice in that variable
356. We will create object inside that slice here we are going to create counter application
357. As this is counter application we write counter login inside reducer
358. Payload ➔ it gets increased the values when we click on increment and goes to logic to increment the value
359. To access the values inside the slice we want to export and we need to get the actions

CounterSlice.jsx:-

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice=createSlice({
  name:counter,
  initialState:{value:0},
```

```
  reducers:{
    increment:state=>{state.value+=1},
    decrement:state=>{state.value-=1},
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    }
 }
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

Step:-3 write in a provider

Store.jsx:-

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from './slices/counterSlice'

const store=configureStore({
    reducer:{
        counter:counterReducer
    }
});

export default store;
```

Step:-4 wrap in main.jsx

main.jsx:-

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { Provider } from 'react-redux'
import store from './redux/Store.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <App/>
    </Provider>
  </StrictMode>,
)
```

Step:-5 we will use in components

360. Here we import useSelector and useDispatcher
361. And we write all our counter application here
362. Inside useSelector by using state we will get the values
363. useDispatcher is for printing the data
364. we must keep this useDispatcher inside the useEffect
365. To print the reducer values functions and to update those values

Counter.jsx:-

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementByAmount } from '../redux/slices/counterSlice';

const Counter = () => {
 const count = useSelector(state => state.counter.value);
 const dispatch = useDispatch();

 return (
  <div>
   <h1>Counter: {count}</h1>
   <button onClick={() => dispatch(increment())}> + </button>
   <button onClick={() => dispatch(decrement())}> - </button>
   <button onClick={() => dispatch(incrementByAmount(5))}> +5 </button>
  </div>
 );
};
export default Counter
```


Fetching Using redux:-

366. We want to keep the fetching login in useEffect
367. We will create extra data here

Day-21/36 Backend of MernStack

Intro:-

Backend:-It is a server side logic

368. MongoDb
369. NodeJS
370. ExpressJs

DNS:- Domain name server/Domain name system

Connection b/w backend to dns:-

371. Request will be sending through client(frontend) that request will be sended to server that is dns server
372. Ex for Dns:- www.google.com
373. Response is sending through backend

How Backend Works/Client server communication:-

374. Req➔goes to domain➔ convert into dns➔ and then to response➔backend ➔ again to Frontend

Why Backend:-

375. To work server side logic
376. For authentication

In Backend we have:-

377. NodeJs
378. Django → Flask
379. .net → c#
380. Spring boot → Java

Why NodeJs:-

381. In node js we are having only one programming language that is js so we are going to use that, if we take any others we want to learn another programming with that
382. Easy to get started
383. Huge job demand

Why MongoDb and why not Sql:-

384. No sql, no structured programming language
385. Used for implementing the real time environment like for deploying or creation
386. Mongodb is very easier than sql
387. In mongodb we are going to create schemas
388. Connection is easy
389. For connection we want to install db connection in sql in mongodb we will have one schema for user for cart we will push that schema into mongodb either compass or atlas
390. We use here mongoose by installing that

ExpressJs:-Framework of NodeJs

NodeJs :- is the library

MongoDb:- is for storing the database to store data

Why ExpressJs instead of NodeJs:-

391. We can do what we are doing in express into node also
392. Eaiser in ExpressJs
393. Code reliability, code reduceability
394. Nodemon for error handling
395. faster

What is Node:-

396. NodeJs is a runtime environment which is used for applying/writing server side logic
397. It allows us to run outside the browser
398. Runtime environment means everything is going to run through the cmd or powershell

NodeJs:-

- It was created in 2009 by Ryan Dahl
- It uses the v8 javascript engine

KeyFeatures of nodeJS:- This is one of interview question:-

- **Non-blocking I/O:-**handles multiple req at the same time
    - NodeJs is most famous for NonBlocking input output statements
- **Event-driven architecture:-**reacts to events like request recived, file read complete
- **Single-threaded:-** one task executes at a time,but very efficient due to the event loop
- **Fast performance:-** because it uses the v8 engine
- **Cross-platform:-** it means working with any platform like windows, macos and linux

Backend Methods :-

399. GET:- to get the data from the backend
400. POST :-to send the data into the backend
401. PUT/PATCH:-to update
402. DELETE:-to delete

Day:-22/36 Of Mern Stack

Project Management(GITHUB):-

For Team Leader:-

403. First create a main repository which is a main branch
404. We want to give collaborate actions➔ go to settings➔ click on collaborators ➔ it asks for password give authentication➔ click on add, people how many people you want to give keep their usernames
405. after adding them they will get mail the people want to click on accept in their mail
406. the mail was sent to people like as below ➔ click on invitation ➔click on accept invitation then you will get their code here

For Team mates:-

407. Who want to do cloning
408. Create new folder ➔ name team folder
409. Open that folder in vsc
410. Write code what the team member want to modify
411. In terminal type git clone (repo link here) ➔enter ➔ we get cloning into 'nec-react'

Git CheckOut:-

412. We want to create branch for that branch we have checkout
413. Git checkout –b (branch name)
414. We will get Switched to new branch (branch name)

After completion of checkout you want to push code into your github

415. git add .
416. git commit -m "clone commit"
417. git push origin prudula

DAY:- 21 of 36 Mern Stack

Deployment using vercel:-

418. Github is not recommended to host react type of projects
419. For this we will use vercel.com➔search vercel.com in browser like as below url shown in the image ➔ click on signup button➔after clicking we will get like as below

420. Click on continue with github➔ after clicking we will get like as the below image in the below there we
421. Want to select for personal or commercial, commercial projects is for business related it will ask money
422. We can click on the personal and enter your name as shown in below image and click on continue

423. click on authorize github,then you will get like as below image

424. Scroll below we will get git install then click on all repistories ➔ and click on install
425. What you want to host you can push that in git and now select that one here

426. Click on import
427. We are using vite project
428. So we don't change anything in framework preset
429. Click on deploy ➔ if deployement is successful ➔ you will get congratulations and scroll down we will get continue to dashboard like as below 2nd image

430. We will get link in domains as shown in above
431. Successfully deployment is completed using vercel

DAY:- 21 of 36 Mern Stack

Deployment using vercel:-

432. Github is not recommended to host react type of projects
433. For this we will use vercel.com➔search vercel.com in browser like as below url shown in the image ➔ click on signup button➔after clicking we will get like as below

434. Click on continue with github➔ after clicking we will get like as the below image in the below there we
435. Want to select for personal or commercial, commercial projects is for business related it will ask money
436. We can click on the personal and enter your name as shown in below image and click on continue

437. click on authorize github,then you will get like as below image

438. Scroll below we will get git install then click on all repistories ➔ and click on install
439. What you want to host you can push that in git and now select that one here

440. Click on import
441. We are using vite project
442. So we don’t change anything in framework preset
443. Click on deploy ➔ if deployement is successful ➔ you will get congratulations and scroll down we will get continue to dashboard like as below 2nd image

444. We will get link in domains as shown in above
445. Successfully deployment is completed using vercel

Day:-22 of Mern Stack

NodeJs:- It is a run time environment which is used for server side scripting logic

Event:- It is a single request and response statement , we can have single or multiple events

Event loop:-It makes iterations every time

446. **Micro Tasks:-** without applying delay (ex:- fetching)
447. **Macro Tasks:-** with applying delay (ex:- set timers)→ Timers→ setTimeout
448. **Call Stack:-**o/p comes to call stack

Event Driven architecture:-

449. Node.js works on events and listeners
450. Instead of following step-by-step instructions, Node.js waits and reacts to events
451. **For Example:-**
    1. When a user sends a request → an event is triggered.
    2. When a file is read → another event is triggered
452. Each event has a callback function that runs when the event occurs.
453. This makes Node.js great for apps that need to handle many users at once(like chat apps or real-time updates)
454. It consists event emmiter , event loop after this we have event handler
455. **Event emmiter:-** it emits all the events and sends to the event loop
456. **Event loop :-** in event loop we have micro and macro tasks and then the output will be sent to the event handler

What can you build with nodeJs:-

457. Web Applications (like APIs)
458. Real-time chat apps (like Whatsapp clone)
459. Restful APIs→ It means creating api from scratch and going to use that
460. Command-line tools
461. Microservices

Core Components of Node.js:-

462. V8 Engine
463. Javascript engine built by Google
464. Compiles Js code to machine code
465. Libuv:-
466. It will be build with c++ and c#

NodeJs Data Flow:-

467. Your code → [V8 Engine] ➔ runs JS →[Libuv] ➔ handles I/O (e.g:- fs,http) → [Thread pool] ➔ offloads long tasks→ [Event loop]➔ waits for task to finish→ Callback➔ runs when task is ready

Initalization of server:-

468. npm init→ it will ask backend package name give that → next asks version it is not mandatory→ description→ command test and some more like this
469. npm init –y (it creates a default package) in terminal
470. I will get by default like as below

471. In the above image we have main in that we have index.js change that to server.js
472. Create a server.js file to write server logic

Creating server in NodeJs:-

Http Server:-An Http server handles requests from clients (like web browsers) and sends back responses.

Why this Server:-

473. Teaches how the backend comminutes with the frontend.
474. Builds a foundation to develop web applications and APIs
475. We have 3000 or 5000 local ports for backend

Creation of server:-

476. For server creation we have a bulitin keyword like createServer
477. We are going to create server inside the server variable in the below example
478. We have another builtin keyword as listen to get the server by using port name

Example server.js (file):-

```
const http=require('http')

const server=http.createServer((req,res)=>{
   res.writeHead(200,{'Content-type':'text/plain'});
   res.end('Hello from Node.js server!');
});

server.listen(3000,()=>{
   console.log("Server is running at http://localhost:3000/");
});
```

479. To get the output run node server.js in the terminal

Modules:-

Modules are reusable blocks of code

480. **Core Modules:-** Built into Node.js (e.g., fs,http)
481. **Local Modules:-**Your own custom files
482. **Third party modules:-**Installed via npm (e.g., express)

Core Modules :-

Example:- By using os we can get our system platform

sever.js:-

```
const os=require('os')
console.log(os.platform())
```

o/p:- win32 (we will get output in our terminal only)

Local modules:-we can create our own functions here

Example:-

math.js:-

```
function add(a,b){
    return a+b;
```

```
}

module.exports=add;
```

server.js:-

```
const add=require('./math');
console.log(add(5,3));
```

o/p:- 8

Third Party Modules:-

moment :- it is used for date functions

- we can get what format we want
- we can also calculate how many days between two dates

To install moment:-

```
npm install moment
```

Example:-1

```
const moment=require('moment')
console.log(moment().format('MMMM Do YYYY, h:mm:ss a'));
```

o/p:- May 22nd 2025, 12:06:26 pm

- Do:- date o is to get nd or th after the number ex:- 2nd or 5th
- a:- am or pm

Example:-2

```
const moment=require('moment')
console.log(moment("2025-12-15","YY-MM-DD").format('MMM D'));
```

Example:-3 To add two months for the current month

```
console.log(moment().add(2,'months').format('YYYY-MM-DD'))
```

Example:-4 To subtract two months from the current month

```
console.log(moment().subtract(2,'months').format('YYYY-MM-DD'))
```

Example:-5 To subtract two dates

```
console.log(moment("2025-05-22").diff(moment("2025-01-01"),'days'));
```

We have some more like

- isBefore
- isAfter
- set locale moment.locale('fr') → to set france date

Curd operations using fs Modules:-

Examples using fs module:-

- This is not only for creating text files it will be for any files like json or another type

Create/Write file:-

```
const fs=require('fs');
fs.writeFileSync('students.txt','Hello Students!');
```

o/p:-when we write we will have a new file in our folder named with students.txt and inside we have the content like Hello Students!

Read File:-

```
const content=fs.readFileSync('students.txt','utf8');
console.log(content)
```

o/p:- in terminal we get Hello Students!

Update file:- To overwrite the previous content we can use writeFileSync or to add to the previous one we can we use appendFileSync

```
const contents=fs.appendFileSync('students.txt','heyy');
const new_content=fs.readFileSync('students.txt','utf8');
console.log(new_content)
```

o/p:- Hello Students! Heyy

Delete file:-

```
const deleteFile=fs.unlinkSync("students.txt");
```

For Mail Transfers:-

- **Install** ➔ npm install nodemailer

- We want to set our mail authentication to set we will have a transport we want to create that transport like as creating server

Example (mailTransfer.js):-

```
const nodemailer=require('nodemailer');

let transporter=nodemailer.createTransport({
    service:'gmail',
    auth:{
        user:'attuluripavanaprudulasri@gmail.com',
        pass:'xfba guzu fjhv wlma'
    }
});

let mailOptions={
    from:'attuluripavanaprudulasri@gmail.com',
    to:'tasneembanu2005@gmail.com',
    subject:'Node js Email Example',
    text:"Hello banu!! What are you doinggg"
};

transporter.sendMail(mailOptions,(error,info)=>{
    if(error){
        return console.log("Error: ",error);
    }
    console.log("Email sent:", info.response);
});
```


- Here we can keep our mail password also but for security purpose we will use our mail app password by generating that
- For sending mail we will create a option called mail option for that we have mailOptions variable

Day:-24 Of Mern Stack

Aggregatoins:-

My orders database is like as below:-

```
db.orders.aggregate([])        (or)        db.orders.find()
```

483. To check what are present in our database we can use any one of them in above commands

To reshape our document (project):-

484. To add new key values or to change the values

```
db.orders.aggregate([{$project:{customer:1,amount:1,discount:{$multiply:["$amount",0.1]},year:{$year:"$date"}}}])
```

485. Customer and amount are already there in our document
486. In the above example customer:1 means applying ascending order for the customer and same as like we are applying for amount also
487. For discount I am applying 10% offer
488. I am getting date from the previous one and adding to the year

UnWind:-Nested array will be removed and the values in that array will come like as separate arrays

```
db.orders.aggregate([{$unwind:"$items"}]
```

For Joins we have lookups here:-

489. We don't have any left or right joins in mongodb because we don't have tables here we are working with documents here everything are like key value pairs
490. **LookUp :-** this for joining the collections

For this we want to create another collection:-

```
db.customers.insertOne({_id:"A",name:"Alice"})
```

Adding customers to the orders collection:-

```
db.orders.aggregate([{$lookup:{from:"customers",localField:"customer",foreignField:"
_id",as:"customer_info"}}])
```

Output:-

Bucket:- To get the data in between boundaries that means in a particular range

```
db.orders.aggregate([
  {$bucket:{
groupBy:"$amount",
boundaries:[0,100,200,300],
default:"Other",
output:{
count:{$sum:1},
orders:{$push:"$_id"}
}
}
}
])
```

Indexes:-

Single Field Index:-

```
db.orders.createIndex({customer:1})
```

Compound Index:-

```
db.orders.createIndex({customer:1,amount:-1})
```

MultiKey Index:-

```
db.orders.createIndex({items:1})
```

Output:-

DAY:-23 Of Mern Stack

MongoDb:- NoSql databse

491. Go to chrome type for mongodb community server download
492. And click on first link to download the mongodb

493. Scroll down click in download
494. Double click on the downloaded file
495. Don’t change anything click on check box and click on next for all upto you get install button
496. After completion of community server install
497. Automatically compass will also be installed
498. While installing mongodb compass we will get like as below image

499. Wait with patience until the bar get completed it will take some more time

500. Automatically it will redirect into mongodb compass like as below

501. Click on add new connection or + (symbol)
502. Next Click on save & continue

503. Click on open mongodb shell for the mongodb
504. By default we have like test> type commands here
505. To check type a simple command show dbs
506. It will give what databases are present by default in that like admin, config, local

507. To use old database or to create new database we will use
    1. Syn:-use databsename
    2. Ex:- use nec

Collection:-Collection of documents is called as collection

DataBase:-Collection of collections is called as a database

508. We want to use one of the object programming language for mongodb
509. To add collections into database

MongoDB Commands:-

To Create Table(Collection):-

510. **Syn:**-db.createCollection(“collection-name”)
511. Ex:- db.createCollection(“users”)

To insert only one object:-

```
db.users.insertOne([{name:"prudulaSri",age:21,cgpa:8.5}])
```

To insert multiple objects:-

```
db.users.insertMany([{name:"prudulaSri",age:21,cgpa:8.5},{name:"rupesh",age:18,cgpa:9.5}])
```

512. We use to .find() to print

For Asc Order:-

```
db.users.find().sort({name:1})
```

For Desc Order:-

```
db.users.find().sort({name:-1})
```

To get only 1 value:-

513. If we will give the number more than the present rows we doesn't get any error,as the output we will get all the values

```
db.users.find().limit(1)
```

To get highest cgpa only 1 row:-

```
db.users.find().sort({cgpa:-1}).limit(1)
```

To search for particular one based on the name:-

```
db.users.find({name:"prudulaSri"})
```

Filter only based on the name:-

```
db.users.find({},{_id:false,name:true})
```

Update (set):-

```
db.users.updateOne({name:"prudulaSri"},{$set:{fullTime:true}})
```

Unset:-

```
db.users.updateOne({name:"prudulaSri"},{$unset:{fullTime:""}})
```

Update Everything in the collection:-

```
db.users.updateMany({},{$set:{fullTime:false}})
```

To update many items:-

514. We have a new key word exists
515. If the old value is false we are going to change that into true

```
db.users.updateMany({fullTime:{$exists:false}},{$set:{fullTime:true}})
```

Delete one item:-

```
db.users.deleteOne({name:"prudula"})
```

Comparison operators:-

516. Less than:-lt

```
db.users.find({age:{$lt:20}})
```

517. Greater than : gt

```
db.users.find({age:{$gt:20}})
```

518. Not equal:-ne

```
db.users.find({age:{$ne:20}})
```

519. Less than equal:- lte

```
db.users.find({age:{$lte:20}})
```

520. Greater than equal :- gte

```
db.users.find({age:{$gte:20}})
```

521. In between:-

```
db.users.find({gpa:{$gte:8,$lte:9.5}})
```

Checks whether present or not:-

522. If we will the name which is not present we doesn't get any error

```
db.users.find({name:{$in:["prudula","rupesh"]}})
```

To get all except particular one:-

```
db.users.find({name:{$nin:["prudula","rupesh"]}})
```

Logical Operators:-

And:- We all know that Both values want to be true for and operator

```
db.users.find({$and: [{fullTime:true},{age:{$lte:25}}]})
```

Or:- We all know that Both values want to be true for and operator

```
db.users.find({$or: [{fullTime:true},{age:{$lte:20}}]})
```

Nor:-Both values want to be false

```
db.users.find({$nor: [{fullTime:true},{age:{$lte:20}}]})
```

Day:-24 Of Mern Stack

Aggregatoins:-

My orders database is like as below:-

```
db.orders.aggregate([])        (or)        db.orders.find()
```

523. To check what are present in our database we can use any one of them in above commands

To reshape our document (project):-

524. To add new key values or to change the values

```
db.orders.aggregate([{$project:{customer:1,amount:1,discount:{$multiply:["$amount",0.1]},year:{$year:"$date"}}}])
```

525. Customer and amount are already there in our document
526. In the above example customer:1 means applying ascending order for the customer and same as like we are applying for amount also
527. For discount I am applying 10% offer
528. I am getting date from the previous one and adding to the year

UnWind:-Nested array will be removed and the values in that array will come like as separate arrays

```
db.orders.aggregate([{$unwind:"$items"}]
```

For Joins we have lookups here:-

529. We don't have any left or right joins in mongodb because we don't have tables here we are working with documents here everything are like key value pairs
530. **LookUp :-** this for joining the collections

For this we want to create another collection:-

```
db.customers.insertOne({_id:"A",name:"Alice"})
```

Adding customers to the orders collection:-

```
db.orders.aggregate([{$lookup:{from:"customers",localField:"customer",foreignField:"
_id",as:"customer_info"}}])
```

Output:-

Bucket:- To get the data in between boundaries that means in a particular range

```
db.orders.aggregate([
  {$bucket:{
groupBy:"$amount",
boundaries:[0,100,200,300],
default:"Other",
output:{
count:{$sum:1},
orders:{$push:"$_id"}
}
}
}
])
```

Indexes:-

Single Field Index:-

```
db.orders.createIndex({customer:1})
```

Compound Index:-

```
db.orders.createIndex({customer:1,amount:-1})
```

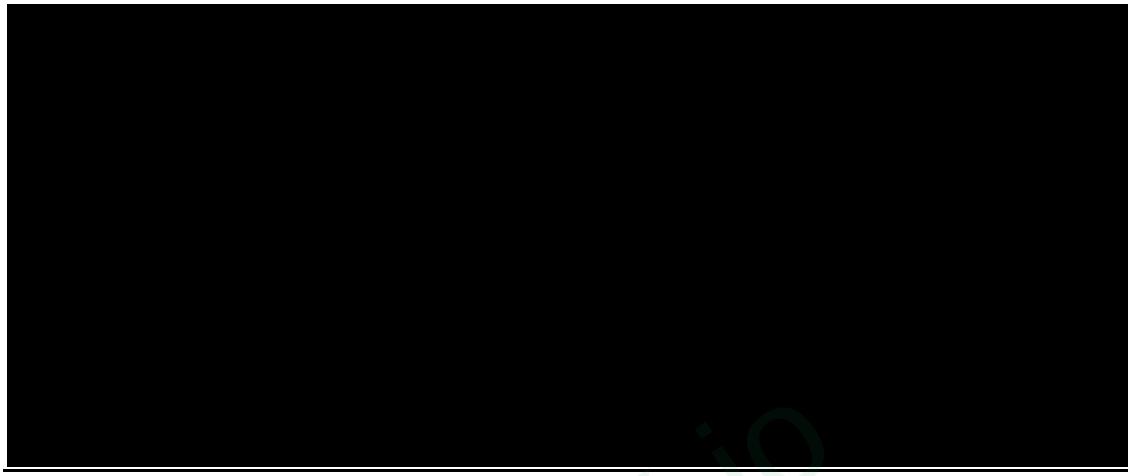
MultiKey Index:-

```
db.orders.createIndex({items:1})
```

Output:-

To get the indexes:-

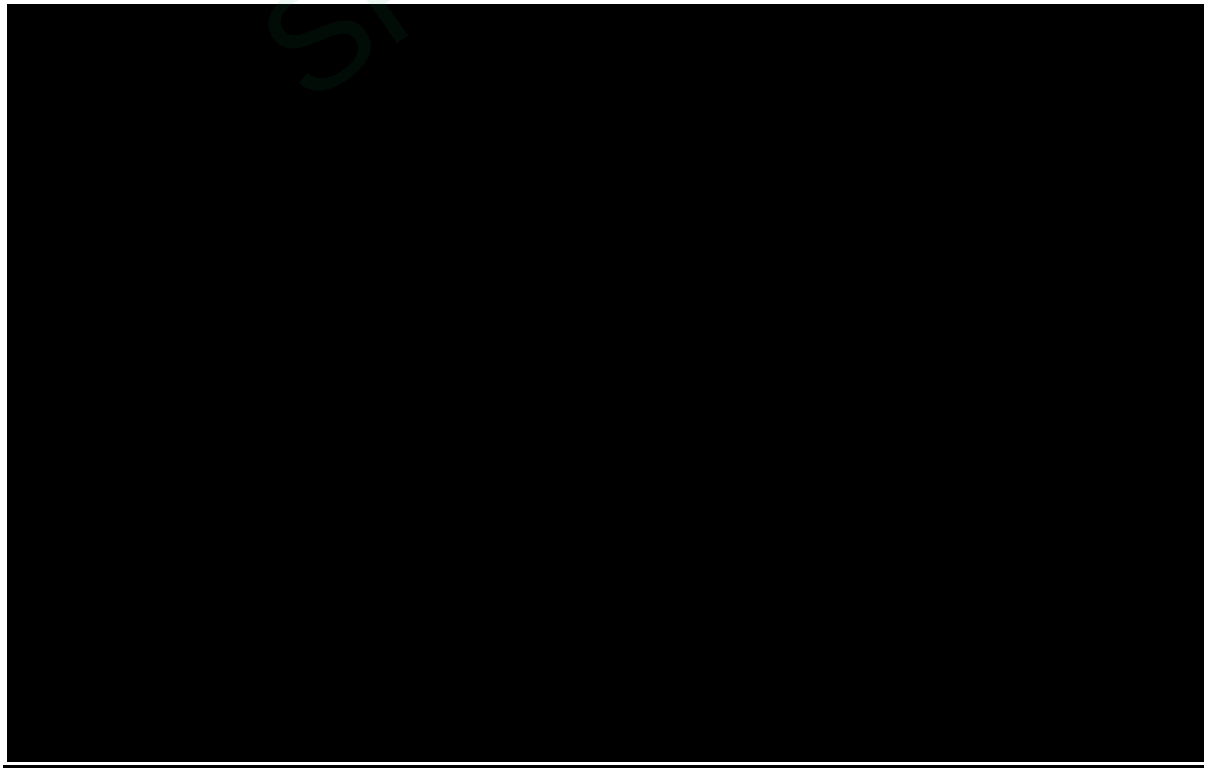
```
db.orders.getIndexes()
```



To check the flow of indexes:-

```
db.orders.find({customer:"A"}).explain("executionStats")
```

- To understand all this we want to learn the total related to mongoDb we will get lots of lines as the output as shown in the below image



DataModelling:-

- Embedded documents approach

Use When:-

- 1:1 connections or 1:few relationships
- Automatic updates are critical
- Read together frequently

Example for joining books with authors:-

Creating authors collection:-

```
db.authours.insertOne({
_id:"auth202",
name:"jane smith",
speciality:"Database systems"
});
```

Creating Books collection:-

```
db.books.insertMany([
{
_id:"book305",
title:"MongoDB Essentials",
author_id:"author202",
},
{
_id:"book306",
title:"Advanced MongoDB",
author_id:"author203",
},
{
_id:"book307",
title:"MongoDb",
author_id:"author204"
}]);
```

- lookup,unwind and project we are using this 3 at a time this process is called as a reference document approach

```
db.books.aggregate([
  {
    $lookup: {
      from: "authors",
      localField: "author_id",
      foreignField: "_id",
      as: "author_details"
    }
  },
  {
    $unwind: "$author_details"
  },
  {
    $project: {
      title: 1,
      "author_details.name": 1,
      "author_details.specialty": 1
    }
  }
]);
```

Schemas:-

- For validations and storing we use schemas

Basic Validation:-

```
db.createCollection("stu", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["employee_id", "full_name", "department"],
      properties: {
        employee_id: {
          bsonType: "string",
          pattern: "^EMP-[0-9]{5}$",
          description: "Must be in format EMP-12345"
        },
        full_name: {
          bsonType: "string",
          minLength: 3,
          maxLength: 25
        },
```

```
      department: {
        enum: ["Engineering", "HR", "Finance", "Marketing"],
        description: "Must be a valid department"
      }

    }
   }
  }
});
```

Output:-

{ ok: 1 }

Insert one pair into that :-

```
db.stu.insertOne({employee_id:"EMP-12345",full_name:"aprudula",department:"HR"})
```

Output:-

- If we will give correct validations we will get acknowledged as true and insertedId object
- If we will give wrong validations we will get document failed validation error

Validation schema for registeration form:-

```
db.createCollection("login_validations", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["username", "email", "password"],
```

```
    properties: {
      username: {
        bsonType: "string",
        pattern: "^[A-Z][0-9][a-z]+$",
        description: "Username must start with a capital letter, followed by a digit, then
lowercase letters"
      },
      email: {
        bsonType: "string",
        pattern: "^[a-zA-Z0-9._%+-]+@gmail\\.com$",
        description: "Must be a valid Gmail address"
      },
      password: {
        bsonType: "string",
        minLength: 3,
        maxLength: 20,
        description: "Password must be between 3 and 20 characters"
      }
    }
  }
 }
})
```

Inserting document:-

```
db.login_validations.insertOne({
  username: "A1abc",
  email: "example@gmail.com",
  password: "securepass123"
})
```

Output:-

Day:-25 of Mern Stack:-

ExpressJs ☺☺ :-

What is ExpressJs:-

531. Express.js is a Node.js web application framework that simplifies building server side applications

Why do we use ExpressJs:-

532. To decrease code length we will use expressjs frame work
533. Easy to use and ease to integrate with backend

Postman:-For checking curd operations we will use this postman

534. Go to chrome and download postman
535. Download → Windows 64-bit
536. After completion of download install that
537. Click all next and install
538. After successful installation we will get like as below

539. In postman we will change the methods and url as shown in below

540. If we observe the below image we have different types of methods like as below get,post,patch,delete,head and options
541. We select what method we want and we will give the url and click on send
542. By using this we will check the data

We need to install ExpressJS:-

543. For installing write npm i express
544. npm init –y to install App
545. For passing the port and getting the result we will write App.listen()

Example1 (server.js):-

```
const express=require('express')
const App=express();
const port=3000;

App.get("/",(req,res)=>{
      res.send("This is my first server")
})

App.listen(port,()=>{
   console.log(`Server is running at http://localhost:${port}`)
})
```

To run :- node server.js

o/p:- This is my first server (we get in the browser)

Output in postman:-

546. To check this output in postman select get method
547. Type url as http://localhost:3000
548. After typing url click on send we will get output in below inside body like as below image

Example2:-

```
App.get('/hello',(req,res)=>{
   res.send("<h1>Hello this message is from hello route</h1>")
})
```

549. Run again and for output type http://localhost3000/hello

o/p:-Hello this message is from hello route

Nodemon:-

550. Here we want to run everytime when we will change the output and want to reload for negotiating this issue we will download nodemon
551. For this npm I nodemon

552. In nodemon for output we will type npm run dev

Example:-3

```
App.get('/info',(req,res)=>{
   console.log("Request Method :",req.method);
   console.log("Request Url:",req.url);
   res.send("Check your terminal for request info");
});
```

o/p:- in chrome browser & in postman we will get output like as below
check your terminal for request info

553. Output in terminal like as below image

Example:-4

```
App.get('/greet',(req,res)=>{
   const name=req.query.name;
```

```
    res.send(`Hello ${name || 'Guest'}`);
});
```

What is Routing:-

Routing in Express means defining how your server responds to different HTTP requests (Get, Post, …etc) for different paths (URLs)

Example:-

```
const express=require('express');
const app=express();
const port=3000;

app.use(express.json());

const students=[
    {id:1,name:"Prudula",course:"Mern Stack"},
    {id:2,name:"Rupesh",course:"Python Full Stack"},
    {id:3,name:"rrr",course:"Mern Stack"},
    {id:4,name:"xxx",course:"Python Full Stack"},
    {id:5,name:"abc",course:"Mern Stack"},
    {id:6,name:"123",course:"Python Full Stack"}
];

app.get("/",(req,res)=>{
    res.send("<h1>Welcome to Student Management</h1>");
})

app.get("/students",(req,res)=>{
    res.json(students);
})

app.listen(port,()=>{
    console.log("running")
})
```

Output in our terminal:-

running

in chrome browser with localhost:3000:- Welcome to Student Management

with localhost:3000/students ➔ is like as below

Output in postman is like as below images:-

To get particular id we will do like as below:-

554. const id = parseInt (req.params.id);

555. req.params.id [redacted]d 's of array of objects stored in variable you specified
556. find() ➔ s = s.id === id (simply to compare something we use find () method)
557. find (s => s.id ===id);
558. s.id means id of objects id
559. what we enter id is stored in variable id and compare with array of object 's id s.id
560. res. status (404) for error

```
app.get('/students/:id',(req,res)=>{
  const id=parseInt(req.params.id);
  const student=students.find(s=> s.id===id);
  student?res.json(student):res.status(404).send("Student not found")
});
```

Output:-

To get name from a particular id :-

```
app.get('/students/:id',(req,res)=>{
  const id=parseInt(req.params.id);
  const student=students.find(s=> s.id===id);
  student?res.json(student.name):res.status(404).send("Student not found")
});
```

Day:-26 of Mern Stack:-

ExpressJs ☺☺:-

✌✌Post Method:-

561. This post method is used to insert new data in json format

❖**Example:-**

```
app.post('/students',(req,res)=>{
   const {name,course}=req.body;
   const newStudent={
      id:students.length+1,
      name,
      course
   };
   students.push(newStudent);
   res.status(201).json(newStudent);
});
```

562. Open postman and select get method and type url like as http://localhost:3000/students
563. Then we get the output what objects we have in that students
564. The output will be like as below

565. Now select post method and click on body in that select raw and json and give a new json object click on send then we will get the object what we enter their as the output
566. The above process is like as shown in below image

🗄⏳⌨📣 Now check your json students data by selecting get method if you observe the newly entered json object will also come as output with the old json students data

✌✌Put Method:-

```
app.put('/students/:id',(req,res)=>{
   const id=parseInt(req.params.id);
   const {name,course}=req.body;
   const student=students.find(s=>s.id===id)

   if(student){
      student.name=name || student.name;
      student.course=course || student.course;
```

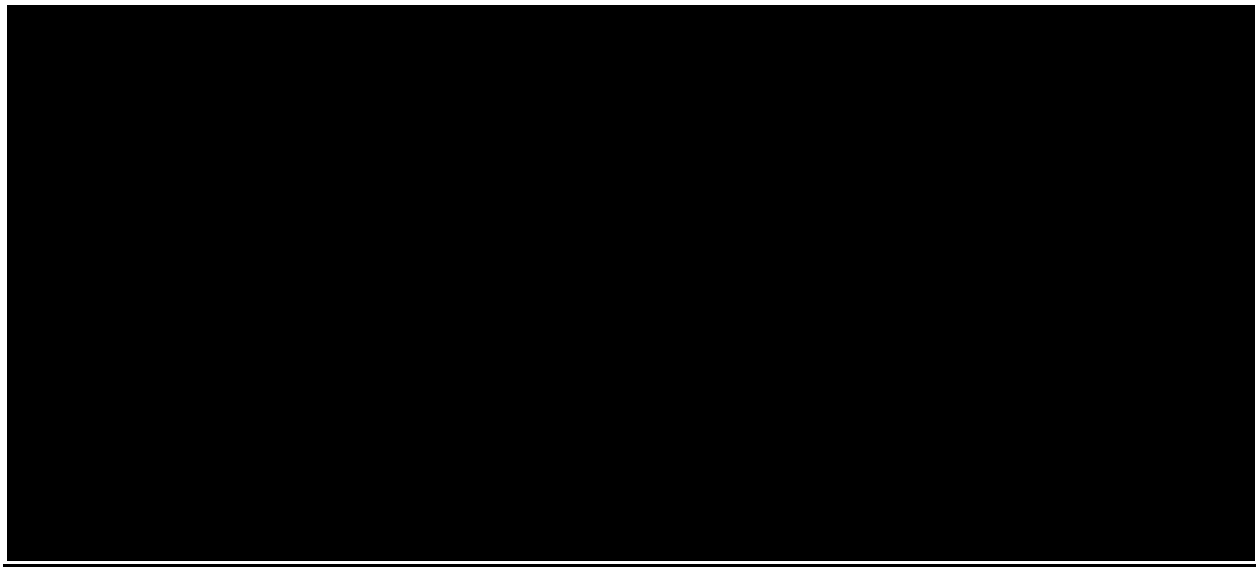
```
    res.json(student);
  }else{
    res.json(404).send("Student not found")
  }
});
```

❖**In postman:-** we will give method as put and url as http://localhost:3000/students/2 (which object you want to update give that id)

Next select method get and observe whether the data is updated or not

✌✌Delete Method:-

```
app.delete('/students/:id',(req,res)=>{
  const id=parseInt(req.params.id);
  const index=students.findIndex(s=> s.id===id);
  if(index!==-1){
    const deleted=students.splice(index,1);
    res.json(deleted[0]);
  }else{
    res.status(404).send("Students not found");
  }
});
```



568. After deleting check again by using get method whether deleted or not

569. Try deleting again then you will get students not found

✌✌**Search:-**

```
app.get('/search',(req,res)=>{
   const {course}=req.query;
   const result=students.filter(s=>s.course===course)
   res.json(result)
})
```

❖**Output in Postman:-**

☞☞What is MiddleWare:-

570. MiddleWare is a function that runs between the client request and the server response

✌Where we use MidleWare:-

571. Log Requests
572. Authenticate users
573. Parse request data
574. Modify request or response
575. Stop or Continue the request cycle

✌Structure of the Middleware:-

```
(req,res,next)=>{....}
```

576. To continue to the next middleware we wil use next

❖Example Logger MiddleWare:-

For debugging we can use this logger and for monitoring, auditing

577. Logs details about each incoming HTTP request to your server
578. HTTP (get, post)
579. Requested URL
580. Response status code 200, 404
581. Time taken to process the request
582. We use this for debugging in real time applications only, no use of this in real time apps
583. next () without exiting it just runs as a loop

Example:-

```
const express = require('express')

const App = express();

const port = 3000;

App.use(logger);


App.get('/search', (req, res) => {

  const { course } = req.query;

  const result = students.filter(s => s.course === course);
```

```
  res.json(result);

});

App.listen(port,()=>{

   console.log(server is running at http://localhost:${port})

})
```

✌✌Urlencoded:-

❖MiddleWares is always put on the top of the code

584. This is used for handling form data

585. This is a built-in middle ware in express used to parse incoming request bodies with url-encoded data

1. Format used by html forms
2. Extended (false):- it parses data using querystring library (limited , simple) this is data limit we don't prefer this
3. Extended(true):- it parses data using the qs library also allows nested objects

❖❖Example:-

```
const express=require('express')
const app=express()
const port=3000

app.use(express.json())
app.use(express.urlencoded({
   extended:true
}));

app.post('/submit',(req,res)=>{
   res.send(`Received:${req.body.name}`);
})

app.listen(port,()=>{
   console.log(`Server is running at http://localhost:${port}`)
})
```

For output:-

586. Open postman select post method and type url like as http://localhost:3000/submit and click on body in that click on form-urlencoded option and below write name and name value as shown in ☝image

Static MiddleWare:-

Example:-

```
const express=require('express')
const app=express()
const port=3000

app.use('/static',express.static('public'));


app.listen(port,()=>{
   console.log(`Server is running at http://localhost:${port}`)

})
```

587. We want to create a folder named with public
588. And inside that folder we want to keep hello.txt

Output:-

Day:-27 of Mern Stack☺☺

MongoDb Atlas:-

589. Go to chrome browser and type mongodb atlas
590. Signin with google
591. After completion of signin you will get like as below

592. In the above image we have seen that we have a folder project 0 click on that we will get drop down in that click on new project
593. Give your project name and click on next and then click on create project
594. Then we will get create a cluster click on create
595. In deploy your cluster click on free
596. And give cluster name mycluster
597. Click on create delyoment

598. Give a project name I am giving like as myProj
599. Click on next

600. Now give a password

601. Click on create database user
602. Click on drivers
603. And copy the below one and click on done

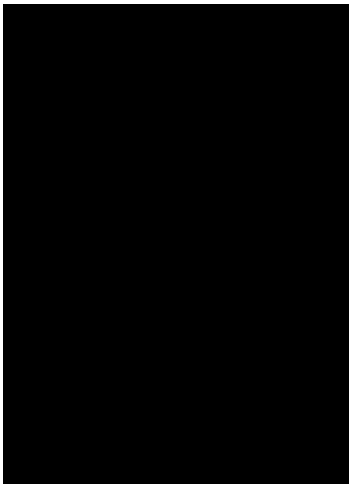
MongoDb Connection url:-

```
mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/?retr
yWrites=true&w=majority&appName=latest
```

Install the following commands:-

604. install mongoose ➔npm I mongoose
605. npm i dotenv
606. npm i mongodb
607. npm i express

Folder structure:-



studentSchema.js:-

```
const mongoose=require('mongoose')
const { type } = require('os')

const studentSchema=new mongoose.Schema({
   name:{
      type:String,
      required:true
   },
   course:{
      type:String,
      required:true
   }
});

module.exports=mongoose.model("Student",studentSchema)
```

.env:-

```
MongoUrl=mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/
?retryWrites=true&w=majority&appName=latest
port=3000
```

server.js:-

```
const express=require('express')
const Student=require('./schemas/studentsSchema')
const mongoose=require('mongoose')
const dotenv=require('dotenv')

dotenv.config()//calling configuration
```

```
const App=express();
App.use(express.json())
const port=process.env.port || 3000

const MongoUrl=process.env.MongoUrl

try{
   mongoose.connect(MongoUrl)
   console.log("mongodb connected")
}catch{
   console.log("mongodb error")
}

App.listen(port,()=>{
   console.log(`The server is running at http://localhost:${port}`)
})
```

o/p:-

Post Method:-

Server.js:-

```
App.post('/students',async(req,res)=>{
   try{
      const student=new Student(req.body)
      const saved=await student.save();
      res.status(201).json(saved);
   }catch(err){
      res.status(400).json({error:err.message});
   }
});
```

o/p in postman:-

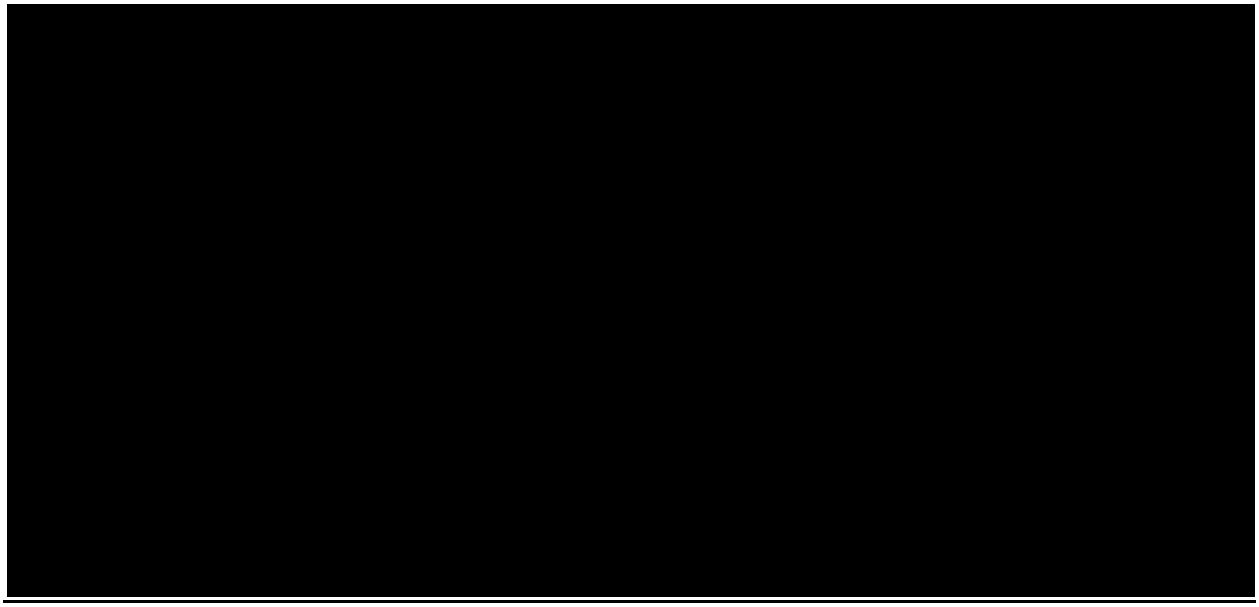
o/p in mongodb:-

608. When we open mongodb we will get like as above image
609. In that click on browse collections,then observe the below image there you will get the data what you inserted in the postman
610. Inside the test in students
611. latest is my cluster name

Get Method:-

```
App.get('/students/:id',async(req,res)=>{
   try{
      const student = await Student.findById(req.params.id);
      if(!student) return res.status(404).json({error:'Student not found'});
      res.json(student);
   }catch(err){
      res.status(400).json({error:err.message})
   }
});
```

o/p:-



Update Method:-

The data will be updated

```
App.put('/students/:id',async(req,res)=>{
   try{
      const updated=await Student.findByIdAndUpdate(req.params.id,req.body,{
         new:true
      });
      if(!updated) return res.status(404).json({
         error:"Student not found"
      });
      res.json(updated);
   }catch(err){
      res.status(400).json({error:err.message})
   }
});
```

Output:-

🙌🙌Day:-28 Of Mern Stack🙌🙌:-

Jwt Tokens:- Tokens for sessions (state and stateless) authentication parts

Rest Api's:- our Api (we are going to post and get the data)

Project structure:-

❖**First step**❖**:-**Install all the dependencies what you want

612. npm init –y
613. npm I express nodemon dotenv mongoose bcryptjs jsonwebtoken

❖**Second step**❖**:-** check in package.json whether they are installed successfully or not

❖**Third Step❖:-**create server

Create server:-

✡**Server.js✡:-**

```
const express=require('express')
const dotenv=require('dotenv')
dotenv.config()
const port=process.env.PORT || 3000
const app=express()

app.use(express.json())

app.listen(port,()=>{
  console.log(`Server is running at ${port}`)
})
```

❖**Fourth Step ❖:-** create schema ,it consists username and password

614. In the below example we want create schema for creating that we will use mongoose.Schema it consists of objects for constructing schemas
615. For username we want to keep that as unique for this we have unique as true and type as String and they must enter this username for that we will keep required as true and we will use trim to remove the spaces
616. For password we don't to keep unique we just keep they must enter the password for that required as true and type as String
617. The code is like as below

```
const userSchema=new mongoose.Schema({
  username:{
    type:String,
    required:true,
    unique:true,
    trim:true
  },
  password:{
    type:String,
    required:true
  }

})
```

618. Here now we want to hash the password before going to the database, we will hash password for security purpose, for hashing we already installed dependency called bycrptjs

619. In this example we will get password by using this keyword and checking whether it is modified or not by using a keyword isModified
620. If the password is not modified we will move to next step
621. If the password is modified then we will hash that password
622. genSalt converts to string format and gets some letters based on the number mentioned,this keyword is just for passing the value
623. we are taking a variable called salt in that we will give in how many values we want to hash the password
624. for hasing we will have the keyword hash inside that we will give what we are going to hash and in how many letters and this hashed password is going to store in this.password variable
625. **syn:-** bcrypt.hash(password,salt) ➔ salt means in many letters they want to hash

```
userSchema.pre('save',async function (next) {
  if(!this.isModified('password')) return next();
  const salt=await bcrypt.hash(this.password,salt);
  this.password=await bcrypt.hash(this.password,salt);
  next();
})
```

❖Fifth Step:-❖In the step we will compare our password and hashed password

By using methods keyword in that we have comparePassword
For comparing we don't use === we will use compare keyword
Export after completion of all this

```
userSchema.methods.comparePassword=function(candidatePassword){
  return bcrypt.compare(candidatePassword,this.password);
};
```

Here is the final User.js code:-

```
const mongoose=require('mongoose')
const bcrypt=require('bcryptjs')

const userSchema=new mongoose.Schema({
  username:{
    type:String,
    required:true,
    unique:true,
    trim:true
  },
  password:{
    type:String,
    required:true
```

```
  }
});

//method to has the password
userSchema.pre('save',async function (next) {
  if(!this.isModified('password')) return next();
  const salt=await bcrypt.hash(this.password,salt);
  this.password=await bcrypt.hash(this.password,salt);
  next();
})

//method to compare password
userSchema.methods.comparePassword=function(candidatePassword){
  return bcrypt.compare(candidatePassword,this.password);
};

module.exports=mongoose.model('User',userSchema);
```

❖Sixth Step:-❖ Connect to mongodb

629. And copy the connection url and keep that in .env file like as below

```
MongoUrl=mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/
?retryWrites=true&w=majority&appName=latest
```

❖Seventh Step:-❖ Connecting mongodb to the server.js

✡User.js✡:-

```
port=3000
const mongoose=require('mongoose')
const bcrypt=require('bcryptjs')

const userSchema=new mongoose.Schema({
  username:{
    type:String,
    required:true,
    unique:true,
    trim:true
  },
  password:{
    type:String,
    required:true
  }
});

//method to has the password
userSchema.pre('save',async function (next) {
```

```
    if(!this.isModified('password')) return next();
    const salt=await bcrypt.genSalt(10);
    this.password=await bcrypt.hash(this.password,salt);
    next();
})

//method to compare password
userSchema.methods.comparePassword=function(candidatePassword){
    return bcrypt.compare(candidatePassword,this.password);
};

module.exports=mongoose.model('User',userSchema);
```

o/p:-

Creating Token:-

630. Whenever we will login a token will be created
631. When you try to login again you can login by using that token, this is for security
632. Import token

✡authMiddleWare.js✡:-

```
const jwt=require('jsonwebtoken')

const authenticate=(req,res,next)=>{
    const token=req.header('Authorization')?.replace('Bearer','');
    if(!token) return res.status(401).json({error:'No token provided'})

    try{
        const decoded=jwt.verify(token,process.env.JWT_SECRET);
        req.user=decoded;
        next();
    }catch(err){
        res.status(401).json({error:'Invalid token'})
```

```
    }
}

module.exports=authenticate;
```

✡.env✡:-

```
MongoUrl=mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/
?retryWrites=true&w=majority&appName=latest
port=3000
JWT_SECRET=attuluripavanaprudulasri123
```

The final files will be like as 👇👇☺☺☺:-

✡authMiddleWare.js✡:-

```
const jwt=require('jsonwebtoken')

const authenticate=(req,res,next)=>{
  const token=req.header('Authorization')?.replace('Bearer','');
  if(!token) return res.status(401).json({error:'No token provided'})

  try{
    const decoded=jwt.verify(token,process.env.JWT_SECRET);
    req.user=decoded;
    next();
  }catch(err){
    res.status(401).json({error:'Invalid token'})
  }
}

module.exports=authenticate;
```

✡User.js✡:-

```
const mongoose=require('mongoose')
const bcrypt=require('bcryptjs')

const userSchema=new mongoose.Schema({
  username:{
    type:String,
    required:true,
    unique:true,
    trim:true
  },
  password:{
    type:String,
```

```
        required:true
    }
});

//method to has the password
userSchema.pre('save',async function (next) {
    if(!this.isModified('password')) return next();
    const salt=await bcrypt.genSalt(10);
    this.password=await bcrypt.hash(this.password,salt);
    next();
})

//method to compare password
userSchema.methods.comparePassword=function(candidatePassword){
    return bcrypt.compare(candidatePassword,this.password);
};

module.exports=mongoose.model('User',userSchema);
```

✡.env✡:-

```
MongoUrl=mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/
?retryWrites=true&w=majority&appName=latest
port=3000
JWT_SECRET=attuluripavanaprudulasri123
```

✡Server.js✡:-

```
const express=require('express')
const dotenv=require('dotenv')
const jwt=require('jsonwebtoken')
const User=require('./models/User')
const authenticate=require('./middleware/authMiddleWare')

const { default: mongoose } = require('mongoose')
dotenv.config()
const port=process.env.PORT || 3000
const app=express()

app.use(express.json())

const MongoUrl=process.env.MongoUrl

try{
    mongoose.connect(MongoUrl)
    console.log(`mongodb connected to string ${MongoUrl}`)
}catch{
    console.log("mongodb connection failed")
```

```
}

app.post('/register', async(req, res, next)=>{
   try{
      const { username, password} = req.body;
      const existing = await User.findOne({username});
      if(existing) return res.status(400).json({error: "User already exists"});

      const user = new User({username, password});
      await user.save();
      res.status(201).json({message: "User registered successfully"});
   }catch(err){
      next(err)
   }
})


app.listen(port,()=>{
   console.log(`Server is running at http://localhost:${port}`)
})
```

Post o/p:-

Login Post:-

```
app.post('/login',async(req,res,next)=>{
  try{
    const {username,password}=req.body;
    const user=await User.findOne({username});
    if(!user|| !(await user.comparePassword(password))){
      return res.status(400).json({error:"Invalid Credentials"});
    }

    const token=jwt.sign(
      {id:user._id,username:user.username},
      process.env.JWT_SECRET,
      {expiresIn:'1h'}
    );
    res.json({token});
  }catch(err){
    next(err);
  }
})
```

LoginPost o/p:-

633. Copy the value of the token

Dashboard:-

Here we are using protected route

```
//protected route
app.get('/dashboard',authenticate,(req,res)=>{
   res.json({message:`Welcome, ${req.user.username}`});
});
```

634. While checking output select get method give url like as
635. And click on headers in that give key as authorization and value as the copied token value

o/p:-

Day:-29 Authentication-Routing😊😊😊

🖐🖐Authentication Router🖐🖐:-

❖❖Folder Structure❖❖:-

✡✡authControllers.js✡✡:-

```
const jwt=require('jsonwebtoken')
const User = require('../models/User')

exports.register=async(req, res, next)=>{
   try{
      const { username, password} = req.body;
      const existing = await User.findOne({username});
      if(existing) return res.status(400).json({error: "User already exists"});

      const user = new User({username, password});
      await user.save();
      res.status(201).json({message: "User registered successfully"});
   }catch(err){
      next(err)
   }
}
```

```
exports.login=async(req,res,next)=>{
  try{
    const {username,password}=req.body;
    const user=await User.findOne({username});
    if(!user|| !(await user.comparePassword(password))){
      return res.status(400).json({error:"Invalid Credentials"});
    }

    const token=jwt.sign(
      {id:user._id,username:user.username},
      process.env.JWT_SECRET,
      {expiresIn:'1h'}
    );
    res.json({token});
  }catch(err){
    next(err);
  }
}

exports.dashboard=(req,res)=>{
  res.json({message:`Welcome, ${req.user.username}`});
};
```

✡✡authRoutes.js✡✡:-

```
const express=require('express')
const router=express.Router()
const {register,login,dashboard}=require('../controllers/authControllers')
const authenticate=require('../middleware/middleware')

router.post('/register',register)
router.post('/login',login)
router.get('/dashboard',authenticate,dashboard)

module.exports=router
```

✡✡User.js✡✡:-

```
const mongoose=require('mongoose')
const bcrypt=require('bcryptjs')

const userSchema=new mongoose.Schema({
  username:{
    type:String,
```

```
        required:true,
        unique:true,
        trim:true
    },
    password:{
        type:String,
        required:true
    }
});

//method to has the password
userSchema.pre('save',async function (next) {
    if(!this.isModified('password')) return next();
    const salt=await bcrypt.genSalt(10);
    this.password=await bcrypt.hash(this.password,salt);
    next();
})

//method to compare password
userSchema.methods.comparePassword=function(candidatePassword){
    return bcrypt.compare(candidatePassword,this.password);
};

module.exports=mongoose.model('User',userSchema);
```

✡✡.env✡✡:-

```
MongoUrl=mongodb+srv://attuluripavanaprudulasri:learn_root@latest.umsx3nt.mongodb.net/
?retryWrites=true&w=majority&appName=latest
port=3000
JWT_SECRET=attuluripavanaprudulasri123
```

✡✡server.js✡✡:-

```
const express=require('express')
const dotenv=require('dotenv')
const jwt=require('jsonwebtoken')
const User=require('./models/User')
const authenticate=require('./middleware/middleware')
const connectDB = require('./db')
dotenv.config()
const port=process.env.PORT || 3000
const app=express()
const authRoute=require('./routes/authRoutes')
```

```
connectDB()

app.use(express.json())

app.use('/auth',authRoute)

app.use((req,res)=>{
   res.status(404).json({error:"Route not found"});
});

app.use((err,req,res,next)=>{
   console.error(err.message);
   res.status(500).json({error:'Server Error'})
})

app.listen(port,()=>{
   console.log(`Server is running at http://localhost:${port}`)
})
```

✡✡db.js✡✡:-

```
const mongoose=require('mongoose')

const connectDB=()=>{
   const MongoUrl=process.env.MongoUrl

   mongoose.connect(MongoUrl)
   .then(()=>console.log("MongoDb Connected"))
   .catch(err=>console.log(err));
}

module.exports=connectDB
```

✡✡middleware.js✡✡:-

```
const jwt=require('jsonwebtoken')

const authenticate=(req,res,next)=>{
   const token=req.header('Authorization')?.replace('Bearer','');
   if(!token) return res.status(401).json({error:'No token provided'})

   try{
```

```
        const decoded=jwt.verify(token,process.env.JWT_SECRET);
        req.user=decoded;
        next();
    }catch(err){
        res.status(401).json({error:'Invalid token'})
    }
}

module.exports=authenticate;
```

o/p:-

Copy token and paste in authorization like the below image

Multer:-

636. We will use this multer to upload videos,files,profile..etc

Step by Step Process:-

♦Step:-1♦

637. Create backend folder and frontend folder separately
638. In backend folder install below ones

♦Step:-2♦

639. npm init –y
640. cors, dotenv , express , mongoose, multer , nodemon

♦Step:-3♦

641. frontend:-
642. npm create vite@latest .

♦Step:-4♦

643. connect with mongodb

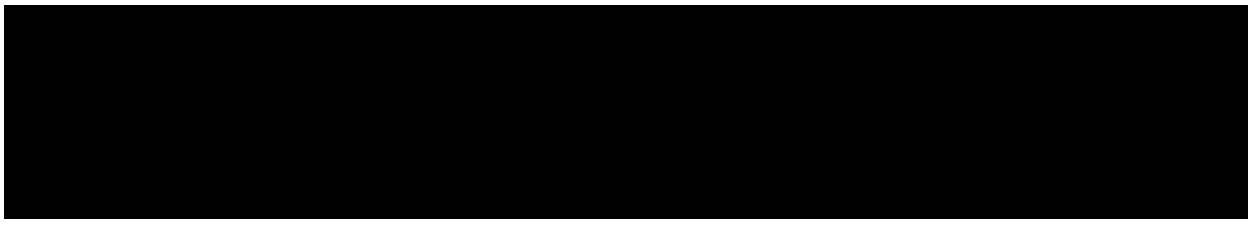
.env:-

```
MongoDb_url=mongodb+srv://attuluriprudula:multer@multercluster.frrrxpl.mongodb.net/?retryWrites=true&w=majority&appName=multerCluster
port=3000
```

.server.js:-

```
const express=require('express')

const app=express()
const mongoose=require('mongoose')
const dotenv=require('dotenv')

dotenv.config()

const MongoDb_url=process.env.MongoDb_url
const port=process.env.port || 3000;

app.use(express.json())

mongoose.connect(MongoDb_url)
  .then(()=>console.log(`MongoDb connected successfully ${MongoDb_url}`))
  .catch(()=>console.log("MongoDb not connected"))

app.listen(port,()=>{
  console.log(`Server running at ${port}`);
})
```

o/p:-



♦Step:-5♦

644. create userSchema
645. for this create models folder and in that create User.js
646. In user.js we are creating userSchema

```
const mongoose=require('mongoose')
const userSchema=new mongoose.Schema({
    name:String,
    email:String,
    dob:String,
    residentialAddress:String,
    permanentAddress:String,
    filePath:String,
});

module.exports=mongoose.model('User',userSchema)
```

♦Step:-6♦

647. Create a folder uploads in that folder we can get our users profile
648. For this we want to create and config multer
649. For this import the below

```
const cors=require('cors')
const multer=require('multer')
const path=require('path')
```

Multer configuration:-

```
const storage=multer.diskStorage({
  destination:'uploads',
  filename:(req,file,cb)=>{
    cb(null,Date.now()+'-'+file.originalname);
  },
});
const upload=multer({storage})
```

♦Step:-7♦

Post:-

650. Now apply curd operations first we will do post request

```
app.post('/register',upload.single('file'),async(req,res)=>{
  try{
    const{
      name,
      email,
      dob,
      residentialAddress,
      permanentAddress,
    }=req.body;
    const filePath=req.file?req.file.path:null;
```

```
        const user=new User({
          name,
          email,
          dob,
          residentialAddress,
          permanentAddress,
          filePath,
        });
        await User.save();
        res.status(201).json({message:'User registered successfully'});
      }catch(err){
        console.error(err);
        res.status(500).json({error:'Server error'});
      }
    });
```

♦Step:-8♦ Get:-

```
    app.get('/users',async(req,res)=>{
      try{
        const users=await User.find();
        res.json(users);
      }catch(err){
        console.error(err);
        res.status(500).json({error:"Server error"})
      }
    });
```

o/p:- Post

Get:-

♦Step:-8♦

651. Now open frontend folder and create registeration form
652. The code is like as below

```
return (
      <div className="registration-card">
     <h2>Register Now</h2>
     <form onSubmit={handleSubmit}>
      <div className="form-group">
       <label className="form-label">Full Name</label>
       <input
        className="form-input"
        type="text"
        name="name"
        value={form.name}
        onChange={handleChange}
        required
       />
```

```
</div>


<div className="form-group">
  <label className="form-label">Email</label>
  <input
    className="form-input"
    type="email"
    name="email"
    value={form.email}
    onChange={handleChange}
    required
  />
</div>


<div className="form-group">
  <label className="form-label">Date of Birth</label>
  <input
    className="form-input"
    type="date"
    name="dob"
    value={form.dob}
    onChange={handleChange}
    required
  />
</div>
```

```
<div className="form-group">
  <label className="form-label">Residential Address</label>
  <textarea
    className="form-input"
    name="residentialAddress"
    value={form.residentialAddress}
    onChange={handleChange}
    required
  />
</div>

<div className="checkbox-group">
  <input
    type="checkbox"
    checked={sameAddress}
    onChange={handleCheckbox}
  />
  <label>Same as Residential Address</label>
</div>

<div className="form-group">
  <label className="form-label">Permanent Address</label>
  <textarea
    className="form-input"
```

```
          name="permanentAddress"
          value={form.permanentAddress}
          onChange={handleChange}
          required
        />
      </div>

      <div className="form-group">
        <label className="form-label">Upload Document</label>
        <input
          className="form-input"
          type="file"
          onChange={(e) => setFile(e.target.files[0])}
          required
        />
      </div>

      <button type="submit" className="submit-btn">
        Submit Registration
      </button>
    </form>
  </div>
);
```

Self Introduction

Good Morning/Afternoon!

I'm delighted to introduce myself — I am **Prudula Sri**, currently in my **final year of B.Tech in Computer Science and Engineering** at **Narasaraopet Engineering College**.

My deep interest in **technology and problem-solving** inspired me to pursue Computer Science as my field of study. Throughout my academic journey, I have maintained a consistent **CGPA of 8.5**.

I have developed a strong foundation in **HTML, CSS, Bootstrap, and JavaScript**, and I also have a basic understanding of **Node.js, Express.js, and MongoDB**. As part of my learning, I built a **portfolio website** using HTML, CSS, and JavaScript. In addition, I've developed several API-based projects including:

653. A **Weather App** using a weather API
654. A **recipe App** using edamam API
655. A **Movie App** using the TMDB API

Currently, I'm working on an **E-learning website**, where I'm integrating the **frontend and backend** to build a complete, dynamic web application.

I am also undergoing intensive training with **Nextwave**. I'm proud to share that I've successfully completed **5 growth cycles** as part of the program, and I'm now starting a new cycle focused on **Data Structures and Algorithms** to strengthen my problem-solving skills further.

Family Background
I come from a **supportive and value-driven family**. My father is a [insert occupation, e.g., government employee / businessman], and my mother is a [insert occupation, e.g., homemaker / teacher]. They have always encouraged me to pursue my goals with dedication and integrity, and their constant support has been my biggest strength throughout my journey.